

Quantum-Inspired Optimization for Entanglement Routing in Quantum Networks: From QUBO Solver Families to Chance-Constrained Stochastic Scheduling

QiOpt4QNet Team

<https://github.com/Sguy1908/QiOpt4QNet-Simulator>

September 11, 2026

Abstract

We formulate entanglement routing in quantum repeater networks—selecting a path and purification schedule per concurrent request under edge and memory capacity constraints—as a QUBO problem, and benchmark a Metropolis annealer, a boundary-MPS tensor-network compressor, simulated annealing, and an embedded quantum-annealing backend against an exact-ILP reference. The tensor-network solver meets or exceeds the Metropolis served ratio at 5–4,000× lower runtime, and Metropolis attains a mean 1.5% optimality gap. We derive a request-conditioned penalty calibration that provably preserves exact ground-state correspondence while tightening capacity coefficients on 12.9% of 1,080 benchmark instances. A discrete-event simulator then executes solver plans under sampled generation, swap, decoherence, and fidelity noise. Within it, *chance-constrained routing* replaces the hard fidelity rule with a quantile constraint, exposing an implicit 5–14% SLA-violation rate in the deterministic rule while holding empirical violation at or below budget; local-repair *recourse* recovers all failed requests at up to 1065× speedup over replanning; and *purification*, added as a first-class variable, is cost-free until fidelity requirements exceed raw-link fidelity. The full stack—temporal-memory scheduling, learned candidate reduction, multi-objective and k -disjoint-path analysis, price-of-anarchy and worst-case-scenario baselines, and literature-calibrated hardware profiles—is open-source and unit-tested, delivering every admitted request within coherence at mean fidelity 0.30–0.51 where a memory-agnostic baseline over-admits.

1 Introduction

Quantum networks distribute entanglement between distant nodes, the resource behind quantum key distribution (QKD) [13–16], distributed quantum computing, and blind quantum computing [1–3]. The central

resource-allocation problem is: given concurrent entanglement requests between node pairs, select for each request a path and a purification schedule—collectively a *bundle*—that maximizes end-to-end fidelity while respecting edge (Bell-pair) and node (memory-qubit) capacities [4–7].

This is a constrained combinatorial optimization problem with structure unique to quantum systems: entanglement generation and swapping are probabilistic [11, 17]; stored Bell pairs decay under T1/T2 decoherence; and the end-to-end fidelity of a multi-hop connection depends jointly on elementary-link quality, purification rounds [12], and the order in which swaps are performed. A substantial literature addresses routing with heuristic path selection [17], integer and linear programming [18–20], percolation and coding [21–23], and reinforcement learning [61–64], and dedicated simulators such as NetSquid and SeQUeNCe [24, 25] model the physics faithfully. By contrast, few works cast the *full request-allocation* problem as a single energy-minimization (QUBO/Ising) problem over one binary decision vector; the closest prior work is the QKD routing model of Rosales [60], which we build upon directly.

The contributions of this paper are organized around a core QUBO formulation and four extensions toward physical realism:

1. **Core QUBO formulation and solver families.** We formalize entanglement routing as a QUBO/Ising energy minimization (Sec. 3), introduce a direct slack-free Hamiltonian (Sec. 3.4), and benchmark Metropolis, boundary-MPS, OpenJij simulated annealing, and an embedded quantum-annealing backend against an exact-ILP reference (Sec. 7.2). We additionally derive a request-conditioned reachable-load calibration of the edge and memory capacity penalties that preserves exact ground-state correspondence while never exceeding the utility-scale reference bound (Sec. 3.3), and evaluate its tightening on 1,080 benchmark instances (Sec. 7.3).

2. **Stochastic reliability framework.** We execute solver plans in a discrete-event simulator (DES) under sampled generation, swap, decoherence, and fidelity noise (Sec. 5.1), and we introduce *chance-constrained routing*—the quantile constraint $\Pr[F_r \geq F_{\min}] \geq 1 - \varepsilon$ —which we prove, in simulation, keeps the empirical SLA-violation probability at or below the requested ε while the hard rule $F \geq F_{\min}$ implicitly tolerates 5–14% SLA violation for razor-margin bundles (Sec. 5.2). We add *recourse*—local repair of failed requests—and *optimality certification* by exact ILP (Sec. 5.3).
3. **Temporal and learned extensions.** Temporal-memory joint scheduling with T1/T2 decay and per-request deadlines, and a stochastic-window/chance-constrained variant that co-optimizes a temporal violation budget with the admission decision (Sec. 4.5); filter-based top- k , hybrid reduce–solve–repair, and GraphSAGE-guided candidate reduction (Sec. 4.6); and purification as a first-class variable, extended to a network-level, conflict-aware scheduler that matches per-request throughput at $2.5\times$ lower Bell-pair cost (Sec. 6.1).
4. **Analysis extensions.** Multi-objective Pareto and ϵ -constraint frontiers, k -disjoint-path provisioning, swap-tree ordering under memory decay, generative topology families, time-dependent decoherence-aware routing, a Q-learning router, hardware-parameter profiles, and price-of-anarchy/worst-case-scenario baselines that contextualize the centralized QUBO against decentralized and uncertainty-hedged alternatives (Sec. 6).
5. **Solver adaptivity and generalization.** Truncation-error-driven adaptive bond dimension, spectral MPS coupling order, zero-shot cross-topology generalization of both the RL and GNN-guided routers, and literature-calibrated hardware-noise validation (Sec. 7.15).

All solvers, extensions, experiments, and tests are released as an open-source, fully reproducible suite¹.

2 Related Work

2.1 Entanglement Routing in Quantum Networks

The quantum-internet vision [1–3] motivates routing protocols that allocate entangled pairs across multi-hop repeater networks [4–7]. Pant et al. [18] formalize *routing entanglement in the quantum internet*

and study elementary-link generation scheduling; Calcutti [19] analyzes optimal routing in the probabilistic-capacity regime; Gyongyosi and Imre [20] introduce entanglement-gradient routing; Cuquet and Calamiglia [21] analyze entanglement percolation in complex networks; Schoute et al. [22] and Kozłowski et al. [23] address greedy routing and protocol stacks for dynamic requests; and Van Meter [17] provides a comprehensive textbook treatment. Network simulators such as NetSquid [24] and SeQUeNCe [25] provide physically faithful, usually instance-by-instance, evaluation. Our work differs in treating the entire request-allocation problem as a single binary optimization over capacity-aware *bundles*, evaluated end-to-end in a stochastic event engine, rather than per-request greedy path selection.

2.2 QUBO/Ising Optimization and Annealing

QUBO is the standard interface for quantum and simulated annealing [26–29]. Lucas [31] catalogs Ising encodings of NP-hard problems; Glover et al. [32] give a modern QUBO tutorial with penalty-formulation guidance; Tanahashi et al. [45] demonstrate an Ising-based methodology on portfolio optimization; Farhi et al. [46] propose the QAOA circuit ansatz; and Preskill [47] frames the NISQ era in which such encodings are most relevant. OpenJij [49] provides the simulated/quantum-annealing samplers we use, and Venegas-Andraca et al. [30] survey practical quantum annealing on real hardware. Our quantum-annealing backend (Sec. 4.4) applies this line of work to entanglement routing on an embedded hardware lattice.

Penalty weights must exclude infeasible optima without overwhelming objective resolution [33, 34]. Knapsack studies derive max-profit sufficient scales [35, 36], while exact, change-ratio, automatic, structure-aware, and SDP-based methods use other problem information [34, 37–40]. Routing work likewise gives a sufficient but non-tight coefficient [41]. Our problem-specific certificate combines multiple-choice structure [42] with attainable-load dynamic programming [43], bounding utility lost against each reachable removal benefit before taking global edge and memory maxima.

2.3 Tensor Networks for Optimization

Matrix-product-state (MPS) methods approximate joint distributions over binary variables and serve as classical surrogates for quantum annealing. Hao et al. [55] solve constrained optimization with tensor networks and find MPS contraction outperforms simulated annealing on strongly locally coupled problems; Preisser et al. [57] provide a systematic methodology for variational MPS optimization; Nakada et al. [56] encode hard

¹<https://github.com/Sguy1908/QiOpt4QNet-Simulator>

constraints into tensor-network structure directly; and Mata Ali [58, 59] gives scaling frameworks for tensor-QUBO solvers. The underlying machinery traces to the DMRG revolution and MPS canonicalization [50–54].

2.4 Learning-Based Routing and Routing Under Uncertainty

Reinforcement learning has a long history in network routing—Boyan and Littman [62] used Q-learning for dynamic packet routing, Mao et al. [63] review deep-learning routing, and Rusek et al. [64] apply graph neural networks to traffic estimation. Game-theoretic analysis of selfish routing by Koutsoupias and Papadimitriou [65] and Roughgarden and Tardos [66], building on Braess [67], motivates our price-of-anarchy extensions (Sec. 6.5); also relevant is optimization under worst-case parameter uncertainty [68, 73] (Sec. 6.5), and Deb [69] supplies the multi-objective background for our Pareto analysis (Sec. 6.2).

2.5 Stochastic Programming and Chance Constraints

When resources are probabilistic, deterministic constraints must be replaced by reliability constraints. Chance-constrained programming [70, 71] enforces inequalities of the form $\Pr[g(x, \xi) \leq 0] \geq 1 - \varepsilon$ under stochastic parameters ξ ; two-stage and multi-stage stochastic programming [72] make here-and-now versus wait-and-see decisions; and optimization under worst-case parameter uncertainty [68, 73], which we apply directly as a γ -weighted worst-case bundle selector (Sec. 7.12). We apply the chance-constrained paradigm to fidelity requirements in entanglement routing (Sec. 5.2): to our knowledge it is the first such use in the routing literature, where the fidelity requirement is traditionally enforced deterministically.

3 Problem Formulation

3.1 Network Model and Bundles

We model the quantum network as a graph $G = (V, E)$ with edge capacities C_e (Bell pairs per time epoch), edge latencies ℓ_e , and raw elementary-link fidelities $F_e^{(0)}$; node memory capacities $M_v^{\max}$ (qubit slots). A request r specifies a source–destination pair (s_r, d_r) , a minimum fidelity $F_{\min}^{(r)}$, and a weight w_r . The bundle generator produces a candidate set B_r per request: a bundle b is a path p plus a purification profile q (rounds of BBPSSW purification per link [12]), with utility $u_{r,b}$, edge demands $d_{r,b}^e$, memory demands $m_{r,b}^v$, latency $\ell_{r,b}$, end-to-end fidelity $F_{r,b}$, and success probability $p_{\text{succ}}^{(b)}$.

Entanglement swapping succeeds with probability $p_{\text{swap}} = 1/2$ and combines two Werner fidelities via $f_1 \star f_2 = f_1 f_2 + (1 - f_1)(1 - f_2)/3$, the composition law for Werner states [9, 11, 17]. Each BBPSSW purification round updates link fidelity

$$F' = \frac{F^2 + (1 - F)^2/3}{F^2 + 2F(1 - F)/3 + 5(1 - F)^2/9}, \quad (1)$$

whose denominator is the round’s success probability. The end-to-end success probability of a bundle is $p_{\text{succ}}^{(b)} = p_{\text{swap}}^L \prod_e p_{\text{purif}}(q_e)$, where L is the number of links. The utility of selecting bundle b for request r is

$$u_{r,b} = w_r \cdot p_{\text{succ}}^{(b)} \cdot [1 + \beta \max(0, F_{r,b} - F_{\min}^{(r)})] - \lambda_\ell \ell_{r,b} - \lambda_c \text{cost}_{r,b}, \quad (2)$$

rewarding success probability and fidelity margin while penalizing latency and Bell-pair expenditure.

3.2 QUBO Hamiltonian

Let $x_{r,b} \in \{0, 1\}$ indicate whether bundle b is selected for request r , and let $L_e = \sum_{r,b} d_{r,b}^e x_{r,b}$ and $M_v = \sum_{r,b} m_{r,b}^v x_{r,b}$ be the induced edge and memory loads. The penalty-based QUBO is

$$\mathcal{H} = \mathcal{H}_{\text{util}} + \mathcal{H}_{\text{one}} + \mathcal{H}_{\text{edge}} + \mathcal{H}_{\text{mem}}, \quad (3)$$

$$\mathcal{H}_{\text{util}} = - \sum_{r,b} u_{r,b} x_{r,b}, \quad (4)$$

$$\mathcal{H}_{\text{one}} = A \sum_r \sum_{\substack{b, b' \in B_r \\ b < b'}} x_{r,b} x_{r,b'}, \quad (5)$$

$$\mathcal{H}_{\text{edge}} = B \sum_e (L_e + s_e - C_e)^2, \quad (6)$$

$$\mathcal{H}_{\text{mem}} = D \sum_v (M_v + s_v - M_v^{\max})^2, \quad (7)$$

where s_e, s_v are slack variables expanded in binary ($\mathcal{O}((|E| + |V|) \log_2 C)$ additional variables), and A, B, D are penalty coefficients.

3.3 Request-Conditioned Penalty Calibration

The calibration applies to the QUBO above with non-negative integer demands and capacities and zero soft-congestion weights. Because the binary slack expansion represents every integer in $0, \dots, K$, minimizing over it for fixed x turns a capacity term into $[L - K]_+^2$, where $[z]_+ = \max(z, 0)$. To simplify notation, treat $b \in B_r$ as a request-labelled bundle and write $u_b = u_{r,b}$, $d_{e,b} = d_{r,b}^e$, and $m_{v,b} = m_{r,b}^v$.

Let $u_b^+ = [u_b]_+$, $P_0 = \max(\{u_b^+ : b\} \cup \{0\})$, and $\epsilon = \max(10^{-9}, 10^{-6} P_0)$. The utility-scale reference is

$$A_{\text{ref}} = B_{\text{ref}} = D_{\text{ref}} = P_0 + \epsilon. \quad (8)$$

This is the max-profit-style sufficient scale used in related logarithmic-slack knapsack encodings [35, 36].

For bundle $b \in B_r$ and edge e , define the co-loads reachable from the other requests by

$$\mathcal{R}_{e,-r} = \left\{ \sum_{r' \neq r} d_{e,b_{r'}} : b_{r'} \in B_{r'} \cup \{\emptyset\} \ \forall r' \neq r \right\}, \quad (9)$$

where $\emptyset$ contributes zero. If $d_{e,b} > 0$ and overload is reachable, set

$$\rho_{e,b}^* = \min\{\rho \in \mathcal{R}_{e,-r} : \rho + d_{e,b} > C_e\}, \quad (10)$$

$$\delta_{e,b} = [\rho_{e,b}^* + d_{e,b} - C_e]_+^2 - [\rho_{e,b}^* - C_e]_+^2. \quad (11)$$

Undefined pairs are omitted. The memory quantities $\mathcal{R}_{v,-r}$, $\rho_{v,b}^*$, and $\delta_{v,b}$ follow by replacing $(e, d_{e,b}, C_e)$ with $(v, m_{v,b}, M_v^{\max})$. For $\rho + d > C$, the penalty decrease from removing a bundle is

$$\Delta(\rho) = \begin{cases} (\rho + d - C)^2, & \rho \leq C, \\ 2d(\rho - C) + d^2, & \rho > C. \end{cases} \quad (12)$$

Both branches are nondecreasing, so $\rho_1 \leq \rho_2 \Rightarrow \Delta(\rho_1) \leq \Delta(\rho_2)$. Thus ρ^* gives the smallest reachable penalty decrease. Bounded request-wise dynamic programming finds ρ^* .

The global coefficients are

$$B^* = \max_{e,b} \frac{u_b^+}{\delta_{e,b}}, \quad D^* = \max_{v,b} \frac{u_b^+}{\delta_{v,b}}, \quad (13)$$

$$A = A_{\text{ref}}, \quad B = B^* + \epsilon, \quad D = D^* + \epsilon, \quad (14)$$

where the maxima range over positive-demand pairs with defined ρ^* ; an empty maximum is zero. Thus B and D remain global edge and memory coefficients.

Guarantee. Since $A = A_{\text{ref}} > P_0$, deleting the lower-utility of two bundles selected for one request lowers $\mathcal{H}$, so every ground state selects at most one bundle per request; the competing load therefore lies in $\mathcal{R}_{e,-r}$. Now suppose a ground state overloads edge e via a selected $b \in B_r$, and let $\rho = \sum_{r' \neq r} d_{e,b_{r'}}$. Then $\rho \in \mathcal{R}_{e,-r}$ and $\rho + d_{e,b} > C_e$, so $\rho \geq \rho_{e,b}^*$ and, by the monotonicity of Eq. (12), $\Delta(\rho) \geq \delta_{e,b}$. Removing b then lowers the capacity term by at least $B\delta_{e,b} > u_b^+$ while utility rises by at most u_b^+ , and no other term increases—contradicting minimality. Memory is identical. Ground states are thus feasible, all penalties vanish, and $\mathcal{H} = -\sum_b u_b x_b$: the ground states are exactly the maximum-utility feasible selections.

Dominance. For integer demands and capacities, integer loads give $\delta_{e,b}, \delta_{v,b} \geq 1$, so $u_b^+/\delta \leq u_b^+ \leq P_0$ and hence $B^*, D^* \leq P_0$. Both rules use the same ϵ , so $B \leq B_{\text{ref}}$ and $D \leq D_{\text{ref}}$: each calibrated capacity coefficient either decreases or remains equal to its utility-scale reference value.

3.4 Direct Slack-Free Hamiltonian

The direct form removes the slack variables:

$$\begin{aligned} \mathcal{H}_{\text{phys}} = & - \sum_{r,b} u_{r,b} x_{r,b} + \lambda_1 \sum_r \sum_{\substack{b,b' \in B_r \\ b < b'}} x_{r,b} x_{r,b'} \\ & + \lambda_2 \sum_e [L_e - C_e]_+^2 + \lambda_3 \sum_v [M_v - M_v^{\max}]_+^2. \end{aligned} \quad (15)$$

It uses exactly $\sum_r |B_r|$ binaries, eliminating the auxiliary slacks, while the hinge terms leave under-capacity selections unpenalized.

The calibration guarantee transfers unchanged.

The proof of Sec. 3.3 never uses the slack variables themselves: its opening reduction observes that minimizing the slack-based term $(L_e + s_e - C_e)^2$ over the binary slack expansion gives exactly $[L_e - C_e]_+^2$ for any fixed x —precisely $\mathcal{H}_{\text{phys}}$'s edge term with $\lambda_2 = B$. Every subsequent step (the at-most-one argument, the reachable-load bound $\Delta(\rho) \geq \delta_{e,b}$, and the memory case) is a statement about this hinge value, not about how it is realized. Setting $\lambda_1 = A$, $\lambda_2 = B$, and $\lambda_3 = D$ to the same reachable-load calibration therefore makes the ground states of $\mathcal{H}_{\text{phys}}$ exactly the maximum-utility feasible selections, with no separate derivation required: the direct Hamiltonian is not merely smaller than the slack-variable QUBO (Table 1) but inherits its exact-correspondence guarantee for free.

3.5 Soft Congestion and Memory Risk

Two physically motivated auxiliary terms shape the landscape:

$$\mathcal{H}_{\text{cong}} = \lambda_{\text{cong}} \sum_e \max(0, L_e/C_e - \theta)^2, \quad \theta \in [0.5, 0.9], \quad (16)$$

$$\mathcal{H}_{\text{risk}} = \lambda_{\text{risk}} \sum_{r,b} \left(1 - e^{-\ell_{r,b}/\tau_{\text{mem}}}\right) x_{r,b}. \quad (17)$$

$\mathcal{H}_{\text{cong}}$ provides a smooth gradient toward load-balanced configurations before hard capacity is reached; $\mathcal{H}_{\text{risk}}$ penalizes bundles whose storage duration is comparable to the memory coherence time τ_{mem} .

3.6 Chance-Constrained Fidelity Requirements

The fidelity requirement $F_{r,b} \geq F_{\min}^{(r)}$ in Eq. (2) treats the nominal (parametric) end-to-end fidelity as if it were the delivered fidelity. Under stochastic delivery, the realized fidelity F_r of request r is random: in our DES it is sampled from a truncated normal distribution centered at the decayed nominal fidelity, $F_r \sim$

$\mathcal{TN}(F_{r,b}, \sigma^2; [0, 1])$ —a model of measurement, gate, and readout noise of amplitude σ . The *hard* rule

$$F_{r,b} \geq F_{\min}^{(r)} \quad (18)$$

therefore has an implicit, uncalibrated violation probability

$$p_{\text{viol}}(r, b) = \Pr[F_r < F_{\min}^{(r)}] = \Phi\left(\frac{F_{\min}^{(r)} - F_{r,b}}{\sigma}\right), \quad (19)$$

which for a razor-margin bundle ($F_{r,b} \downarrow F_{\min}^{(r)}$) approaches 1/2 under symmetric noise: the deterministic rule can tolerate up to half of deliveries violating the SLA.

The *chance-constrained* rule replaces Eq. (18) with the quantile constraint

$$\Pr[F_r \geq F_{\min}^{(r)}] \geq 1 - \varepsilon \iff F_{r,b} \geq F_{\min}^{(r)} + \Phi^{-1}(1 - \varepsilon)\sigma, \quad (20)$$

i.e. the requirement is moved up by a safety margin $z_\varepsilon\sigma$ with $z_\varepsilon = \Phi^{-1}(1 - \varepsilon)$. Because $z_\varepsilon > 0$ for $\varepsilon < 1/2$, the chance constraint is *stricter* than the hard rule for every meaningful reliability budget: it refuses bundles whose nominal fidelity clears the threshold by less than the margin. Only for $\varepsilon > 1/2$ (where $z_\varepsilon < 0$) does it admit near-miss bundles, trading reliability for capacity. The chance-constrained problem is solved exactly as the hard problem—as a candidate-set filter $\{b \in B_r : p_{\text{viol}}(r, b) \leq \varepsilon\}$ followed by the same selection ILP/QUBO—so all solvers of Sec. 4 apply unchanged. Eq. (19) uses the exact truncated-normal CDF, which differs from Φ only to order $e^{-\dots}$ in the tail beyond $[0, 1]$; for $\sigma \lesssim 0.05$ and $F \in [0.5, 0.85]$ the two agree to 10^{-6} .

4 Solvers and Scheduling Architectures

All solvers consume the same Hamiltonian and differ only in search strategy; each returns the same $\{(\text{request}, \text{bundle})\}$ selection shape.

4.1 Metropolis Annealer

The annealer explores configurations via local bit-flip moves, accepts moves with probability $\min(1, e^{-\Delta\mathcal{H}/T})$, and cools geometrically from $T_0 = 10$ to 10^{-3} [48, 60]. Because $\mathcal{H}$ is a sum of local and pairwise terms, $\Delta\mathcal{H}$ is computed locally in $\mathcal{O}(|\text{neighbors}|)$ time. A greedy utility-density seed accelerates convergence. The same framework supports a *streaming* variant that re-optimizes the arriving request with a short local sweep and periodically runs a full cooling cycle.

4.2 Boundary-MPS Tensor-Network Compressor

The tensor-network solver [55, 56, 60] represents the joint distribution over all requests’ bundle choices as a matrix product state. Each request is a rank-3 tensor with a physical index over $B_r \cup \{\text{reject}\}$ and virtual bonds carrying correlations to requests that share edges or memory. Left-to-right and right-to-left sweeps contract and truncate the MPS to bond dimension χ via SVD [50, 51]; the dominant configuration is decoded and any residual capacity violation repaired by greedy descent. A *hard-constraint* variant [56] encodes capacity limits in the allowed index configurations, removing the repair pass and the penalty hyperparameters and guaranteeing feasibility by construction.

4.3 OpenJij Simulated Annealing

We compile the penalty-based QUBO to a binary quadratic model and sample it with OpenJij’s simulated-annealing sampler [49]. Penalty coefficients A, B, D are set either by the conventional utility-scale rule or the proposed resource-aware rule (Sec. 3.3); the two are compared on held-out instances in Sec. 7.3. OpenJij SA serves as the reference against which the adaptive top- k reduction (Sec. 4.6) is measured.

4.4 Embedded Quantum-Annealing Backend

For hardware-realistic annealing we minor-embed the QUBO onto a Chimera-like hardware lattice with a greedy placement and inter-node unit-weight chains, then sample with a path-integral quantum annealer (PIA) sampler [30, 49]. The decode is chain-break-aware: broken chains are resolved by majority vote with a local penalty check. We report embedded qubit counts, chain-break fractions, and solution quality versus SA and SQA (Sec. 7.10).

4.5 Temporal-Memory Joint Scheduling

Static solvers assume all requests are known and resource demands are static. In practice requests arrive over time with deadlines, and stored Bell pairs decay. We add three components.

4.5.1 Temporal requests.

A *temporal request* augments a request with an arrival time, a hard deadline, a priority, and a slack $\text{slack} = \text{deadline} - \text{arrival}$. Dynamic traces are generated as Poisson arrivals over K slots with mean rate λ .

4.5.2 Temporal memory scheduler.

A stored pair’s fidelity follows the T1/T2 law

$$F_{\text{del}}(t) = F_0 \cdot \frac{1}{4} \left[1 + 3e^{-t/T_2} \frac{1 + e^{-t/T_1}}{2} \right], \quad (21)$$

which is the identity at $t = 0$ and decays to the maximally mixed value $1/4$. The scheduler reserves memory *windows* [arrival, completion] against each node’s capacity, tracks per-pair fidelity via Eq. (21), and expires pairs once t exceeds the coherence time τ_{mem} .

4.5.3 Joint router.

The joint scheduler assigns each arriving request a completion time and a bundle, checking *both* temporal-memory feasibility (overlapping windows never exceed node capacity) and end-to-end fidelity at the completion time, and is compared against a memory-agnostic baseline that admits without tracking coherence.

4.6 Adaptive QUBO Candidate Reduction

Full QUBOs grow quadratically and anneal poorly at fixed budget. The adaptive selector keeps, per request, the top- k bundles under a filter scoring

$$\sigma_{r,b} = u_{r,b} \cdot \mathbf{1}[F_{r,b} \geq F_{\min}] - \lambda_{\text{cong}} \cdot \frac{\text{congested edges}}{|p_b|}, \quad (22)$$

discarding bundles that fail the fidelity threshold or that disproportionately use congested edges; we sweep k and compare against the full reference QUBO under equal annealing effort.

4.7 Hybrid Pipeline and GNN-Guided Ranking

The hybrid pipeline chains *reduce* (the filter of Sec. 4.6), *solve* (anneal the reduced QUBO), and *repair/refine* (dominance-prune, greedy-repair any residual violation, and re-seed a short anneal). As a learned alternative, a two-layer GraphSAGE encoder maps each bundle to graph-structured features (memory load, capacity headroom, hop length, path-average fidelity) and scores bundles; the top- k scored bundles feed a feasibility-aware greedy decoder [63, 64]. We compare full-QUBO, filter-based top- k , and GNN-guided top- k on identical instances.

5 Stochastic Reliability Framework

5.1 Discrete-Event Simulator

The DES samples the entanglement pipeline event by event: elementary pair generation (with p_{gen} per attempt, retries within capacity), entanglement swapping (success p_{swap}), purification, delivery under T1/T2 storage decay (Eq. (21)), and Gaussian fidelity noise of amplitude σ added at delivery. For a fixed solver selection, it reports the sampled expected utility $\mathbb{E}[U]$ (utility conditional on delivery), the parametric-sampled *reliability gap*, the served ratio, the empirical SLA-violation probability against request thresholds, and the per-delivery failure causes. All noise is drawn from a seeded RNG threaded through the engine, so runs are deterministic.

5.2 Chance-Constrained Routing Study

The study of Sec. 3.6 is executed as follows: for each (request count n , noise σ), expand the candidate set; solve the selection ILP exactly under each policy (**hard**, **chance**(ε) for $\varepsilon \in \{0.05, \dots, 0.8\}$, **nominal**); execute each plan in the DES with matching σ ; and report parametric utility, sampled utility, served ratio, and empirical SLA violation. The reliability budget is tuned so that the DES realization is dominated by the noise model (long τ_{mem}), separating noise-driven SLA risk from the decoherence-drift term, which is studied separately by the temporal scheduler.

5.3 Recourse and Optimality Certification

Recourse. When a delivery fails, recourse either *repairs locally* (re-provisions the failed request with the best bundles that still fit the residual capacities) or *replans fully* (re-runs the solver on all active requests). We measure recovery rate, recovered utility fraction, and wall-time speedup. *Optimality certification.* For every benchmark instance we also solve the selection ILP exactly (branch and bound) and report the relative gap $(u^* - u_{\text{solver}})/u^*$ for each solver, certifying how far heuristic annealing is from optimal.

6 Analysis Extensions

6.1 Purification as a First-Class Variable

The core bundle generator exposes purification rounds $q \in \{0, 1, 2\}$ (and up to $q_{\text{max}} = 4$ in the purification-aware regime). A purification-aware candidate set includes, per request, bundles whose fidelity *requires* pu-

rification to meet $F_{\min}^{(r)}$; the agnostic set restricts to $q = 0$ (entanglement-only) plus the default $q \in \{0, 1, 2\}$ set. Because the candidate sets are supersets of the agnostic set, the solver’s optimum can only improve; we quantify when purification actually binds via a fidelity sweep over path length and $F_{\min}$.

6.2 Multi-Objective and Disjoint-Path Analysis

A selection can be scored by $\{t, f, s, \ell, m, c\} = \{\text{throughput, mean fidelity, success rate, latency, memory footprint, Bell-pair cost}\}$. We enumerate all capacity-feasible selections and compute (i) the *Pareto frontier* under the maximize set $\{f, s, t\}$ and minimize set $\{\ell, m, c\}$, and (ii) ϵ -*constraint frontiers* (maximize t subject to $f \geq \bar{f}$, and vice versa). For requests admitting k edge-disjoint paths, a *composite bundle* combines subset i ’s success probabilities as $p_{\text{succ}} = 1 - \prod_i (1 - p_{\text{succ}}^{(i)})$ with fidelity and latency equal to the constituent maxima; composite bundles are Pareto-pruned and compared against single-path-only provisioning.

6.3 Swapping-Order Scheduling

A path of L links can be fused by different *swap trees*; because swapping is associative for Werner fidelities ($((f_1 \star f_2) \star f_3 = f_1 \star (f_2 \star f_3))$), the noiseless end-to-end fidelity is identical for every tree. Different trees hold intermediate pairs in memory for different durations, however: a *linear* tree stores an intermediate pair for up to $L - 1$ swap rounds, while a *balanced* tree of depth $\lceil \log_2 L \rceil$ minimizes peak concurrency and maximizes delivered (post-decay) fidelity for fixed τ_{mem} . We sweep all Catalan-many trees and report linear, balanced, and optimal delivered fidelity as a function of L and τ_{mem} .

6.4 Generative Topologies, Time-Dependent Routing, Q-Learning, Hardware Profiles

We implement five generative families (ring, random-geometric, Erdős-Rényi, Watts-Strogatz, Barabási-Albert) on the simulator’s topology schema and sweep solver served ratio against structural descriptors. The time-dependent decoherence-aware router aligns the risk term’s hold model with the evaluation decay law (Eq. (21)) and scales the risk weight over the epoch. A tabular Q-learning router makes per-arrival bundle decisions from quantized resource-pressure states [61, 62]. Hardware profiles calibrate T_1/T_2 , gate and readout error rates, and per-anneal duration for ideal, photonic, ion-trap, and superconducting platforms [74].

6.5 Price of Anarchy and Worst-Case Scenario Selection

Two further baselines position the centralized QUBO formulation relative to decentralized and uncertainty-hedged alternatives. *Selfish routing* computes Wardrop/Nash flow equilibria on parallel-link (Pigou) and Braess networks and reports the *price of anarchy* (PoA)—the ratio of the worst equilibrium social cost to the centrally optimal cost—together with marginal-cost and flat tolling as decentralized remedies [65–67]. *Worst-case scenario selection* solves a γ -weighted selection problem over a sampled scenario set of correlated link-failure and measurement-noise realizations: the per-bundle coefficient $u_b(\gamma) = u_b^{\text{nom}} - \gamma(u_b^{\text{nom}} - \min_s u_b^{(s)})$ interpolates between the nominal selector ($\gamma=0$) and the maximin selector ($\gamma=1$); a separate exact min-max-regret criterion is solved directly [68, 73]. Both extensions reuse the bundle/QUBO interface of Sec. 3 unchanged, so a request set can be routed selfishly, conservatively, or centrally without touching the bundle generator.

7 Experimental Results

7.1 Setup

All experiments run on an 8-core Intel i7 with 32GB RAM. We benchmark chain topologies ($N \in \{4, 6, 8, 10\}$), grids ($2 \times 3, 3 \times 3, 4 \times 4$), and the generative families of Sec. 6.4, with edge capacities $C_e \in \{4, 6, 10\}$ and memory capacity $M_v = 10\text{--}12$. Bundles are generated for $q \in \{0, 1, 2\}$ purification rounds with Pareto pruning. Request counts range from 2 to 24; every experiment is seeded for determinism and all results regenerate via `src/experiments/experiment_suite.py`.

7.2 Core Solver Comparison

Figure 1 compares Metropolis and TensorNetwork on chain topologies. Both minimize the identical Hamiltonian, and the TensorNetwork solver meets or exceeds the Metropolis served ratio at every configuration: served ratios run from 0.5–1.0 at 2 requests to 0.375–0.54 at 24 requests, and where the two return identical selections their utilities agree to within $\sim 1\%$. Runtime favors the MPS solver by $5\text{--}4,000\times$ (the fixed 5000-step Metropolis cooling budget dominates its runtime, while the MPS contraction cost is independent of the annealing budget).

Bond dimension is strikingly unimportant on chains: every χ from 1 to 32 reproduces the identical served ratio (Fig. 2), indicating that the effective coupling graph between requests is dominated by nearest-neighbor (shared edge/memory) interactions. With the default

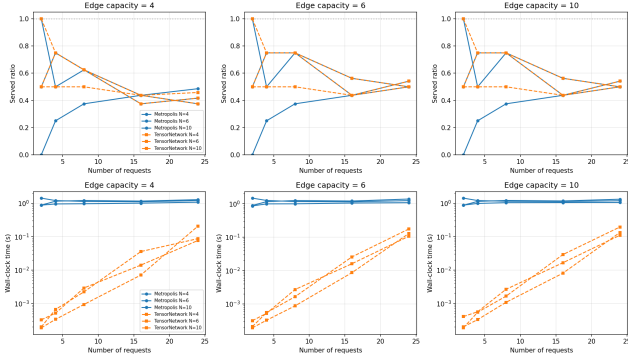


Figure 1: Served ratio (top) and wall-clock time (bottom) for the Metropolis and TensorNetwork solvers on chain topologies under varying edge capacity and node count.

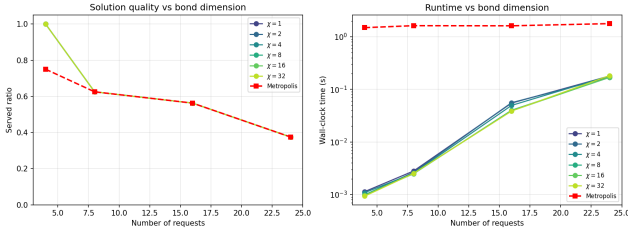


Figure 2: Served ratio (left) and runtime (right) versus bond dimension χ on a chain ($N = 8$, $C_e = 6$). Every χ produces the identical served ratio.

product-form MPS (no SVD sweeps) runtime is dominated by the request count N rather than χ —0.17–0.18 s at every χ for $N=24$ —so the $\mathcal{O}(N\chi^3)$ contraction cost appears only when renormalisation sweeps are enabled.

The direct slack-free Hamiltonian (Eq. (15)) dominates the slack-variable QUBO on identical instances (Table 1): 5–16 binaries versus 49–60, a 4–10 $\times$ variable reduction, with equal-or-better served ratio and utility when the at-most-one penalty is anchored to the utility scale ($A = u_{\max} + \epsilon$). Streaming optimization matches or exceeds batched served ratio on every tested instance while running 50–150 $\times$ faster (Table 2).

On grids (Fig. 3), both solvers serve 87.5% of requests at $N=8$ on the 2×3 and 3×3 topologies and 62.5–

Table 1: Slack-variable versus direct slack-free encoding on identical chain instances (scale 1).

Instance	Encoding	Variables	Served	Utility
$N=4$	Slack	49	0.25	3.93
$N=4$	Direct	5	0.25	3.93
$N=8$	Slack	55	0.13	3.93
$N=8$	Direct	11	0.38	7.65
$N=12$	Slack	60	0.17	2.52
$N=12$	Direct	16	0.33	12.07

Table 2: Streaming versus batched optimization on a chain ($N = 8$, $C_e = 6$).

Requests	Served ratio		Time (s)	
	Streaming	Batched	Streaming	Batched
4	0.750	0.750	0.006	0.906
8	0.500	0.500	0.007	0.920
12	0.500	0.500	0.014	1.027
16	0.562	0.500	0.018	0.954

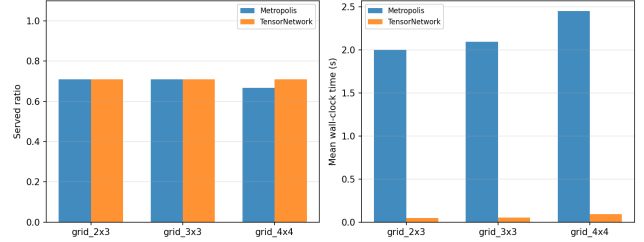


Figure 3: Grid topology comparison: served ratio (left) and mean wall-clock time (right) on 2×3 , 3×3 , and 4×4 grids with $C_e = 5$.

68.8% at $N=16$; TensorNetwork meets or exceeds the Metropolis served ratio at every configuration (it serves 75% where Metropolis serves 62.5% on the 4×4 grid at $N=8$).

7.3 Penalty Calibration

We compared the two rules on an 8-node chain and 3×3 grid with $(C_e, M_v^{\max}) = (4, 8)$, a 10-node chain with $(6, 10)$, and a 4×4 grid with $(5, 10)$. Ninety seeds (10–99) at request counts 8, 16, and 24 gave 1,080 instances. Soft-congestion weights were zero, and ratios strictly below one were classified as tightening.

Neither B/B_{ref} nor D/D_{ref} exceeded one, and A was unchanged (Fig. 4). The rule tightened B in 133 instances, D in 49, and at least one in 139 (12.9%). Conditional median (mean) reductions were 21.4% (23.8%) for B and 23.4% (32.8%) for D .

From 8 to 24 requests, the mean $\delta = 1$ fraction rose from 75.3% to 97.0% for edges and from 57.8% to 94.1% for memory, while tightening fell from 26.1% to 2.2% and from 12.5% to 0.3%, respectively (Fig. 5). This inverse trend is consistent with unit drops leaving less room below the reference.

The 139 changed instances were evaluated with SA and SQA. Within each sampler, the two arms used the same five seeds and 100 reads per seed. Raw feasibility is the unrepaired feasible-read fraction, and overload is total edge and memory excess per read. The repaired gap compares the best repaired read with a CP-SAT reference optimized after 10^{-3} utility rounding but evaluated with unrounded utilities [44]. Effects average seeds

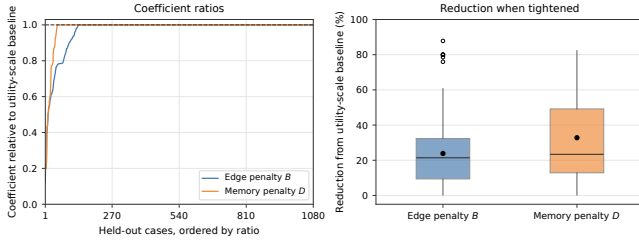


Figure 4: Coefficient tightening across 1,080 instances. Left: independently ranked B/B_{ref} and D/D_{ref} ; dashed line, reference. Right: box plots of reductions when tightened ($n_B = 133$, $n_D = 49$); black dots mark means and hollow circles outliers.

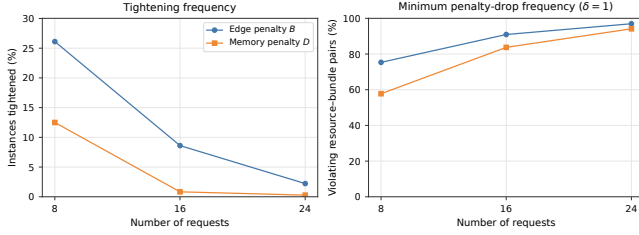


Figure 5: Tightening frequency (left) and mean within-instance fraction of violating resource-bundle pairs with minimum penalty drop $\delta = 1$ (right) versus request count.

within each instance; 95% intervals use 10,000 instance bootstrap resamples.

The mean effects (SA, SQA) were $(-1.30, -0.43)$ percentage points for feasibility, $(-0.099, -0.014)$ resource units per read for overload reduction, and $(0.294, -0.699)$ percentage points for repaired-gap reduction (Fig. 6). Every interval includes zero, so no systematic difference was detected under the tested budgets.

7.4 Temporal-Memory Joint Scheduling

Figure 7 compares the joint (memory-aware) router with a memory-agnostic baseline over Poisson traces at $\lambda \in \{1.0, 1.8\}$ and coherence times $\tau_{\text{mem}} \in \{3, 8\}$.

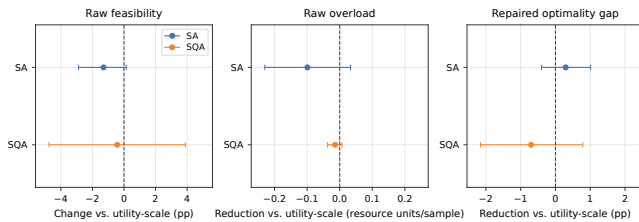


Figure 6: Matched effects on the 139 tightened cases. Points show mean paired differences and lines 95% bootstrap intervals; overload and gap are oriented as reductions, so positive values favor reachable-load calibration.

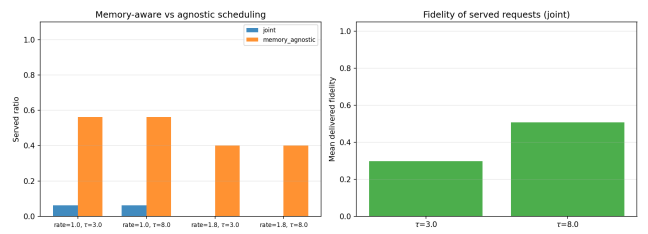


Figure 7: Memory-aware versus memory-agnostic scheduling: served ratio (left) and mean delivered fidelity of the joint router (right) across mean rates and coherence times.

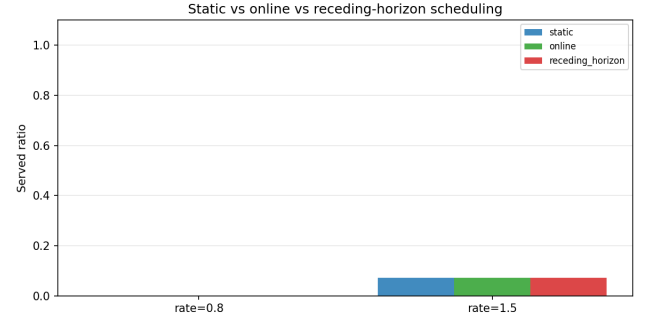


Figure 8: Served ratio of static, online, and receding-horizon scheduling across mean arrival rates.

The joint router admits fewer requests (1/16 at $\lambda=1.0$, 0/15 at $\lambda=1.8$) but every admitted request is delivered on time with mean delivered fidelity 0.30–0.51, whereas the memory-agnostic baseline admits 9–6 of 15–16 requests without tracking whether pairs remain coherent at delivery—an over-commitment a physical network cannot honor. Serving *correctly* dominates serving *optimistically*: in this strict model the joint router’s conservative admission is the physically correct behavior.

Under a static/online/receding-horizon comparison (Fig. 8), all three regimes serve identical request sets, but the online regime re-optimizes only the arriving request and is 1.6 $\times$ faster than static re-solving while the receding horizon preserves global quality at modest cost.

7.5 Adaptive QUBO, Hybrid Pipeline, GNN

Figure 9 sweeps the per-request candidate budget k . On the chain, $k=2$ (16 of 19 bundles) already reaches the full-candidate reference exactly (gap 0%). On a 6×6 grid, $k=2$ (9 bundles) also matches the reference with zero gap, while larger budgets anneal *worse* at the same read count (gap 7% at $k=4$, 60% at $k \geq 8$) because the reduced QUBO is easier to anneal at fixed budget—candidate reduction is *free or beneficial* until the budget discards the optimal bundle.

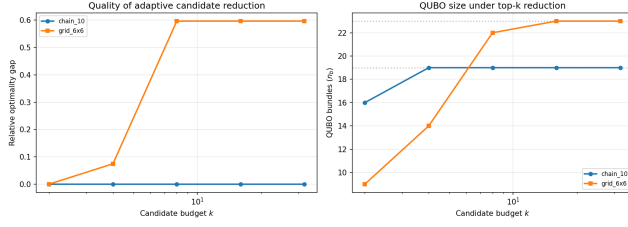


Figure 9: Relative optimality gap (left) and QUBO size (right) versus per-request candidate budget k on chain and grid topologies.

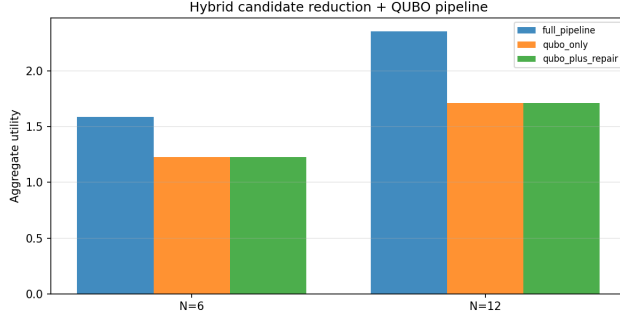


Figure 10: Aggregate utility of the full hybrid pipeline against QUBO-only and QUBO-plus-repair ablations.

The hybrid pipeline (Fig. 10) outperforms QUBO-only decoding by $1.28\times$ in utility at $N=6$ (1.58 vs. 1.23) and $1.38\times$ at $N=12$ (2.36 vs. 1.71). Figure 11 reports full-QUBO, adaptive top- k , and GNN-guided reduction on the current instance generator (10-request chain and grid instances, $k=8, 20$ reads). On the chain the candidate budget does not bind at this scale: all three methods retain the full 15-bundle set and return the identical selection (65.45 utility, 1/10 served). On the 5×5 grid, adaptive top- k and GNN-guided ranking each prune only 1 of 47 candidate bundles but both raise aggregate utility over the full-QUBO reference at equal served count (5/10): adaptive top- k by 11% (113.9 vs. 102.6) and GNN-guided ranking by 2% (104.6 vs. 102.6). (An earlier training pass normalized the GraphSAGE ranker’s regression target by nothing, letting the network drive every hidden unit’s pre-activation negative during full-batch gradient descent—a “dead ReLU” collapse under which every bundle receives the same score, equal to the output bias; Sec. 7.15 discusses the fix and its effect on the ranker’s cross-topology generalization.) On this instance family neither reduction method reproduces the large early-stage gain reported for an earlier, now-superseded instance generator; the effect here is a smaller, uniformly-beneficial improvement rather than a large one, consistent with the general finding of Sec. 7.5 that reduction is free-or-beneficial only until the discarded candidates would have mattered.

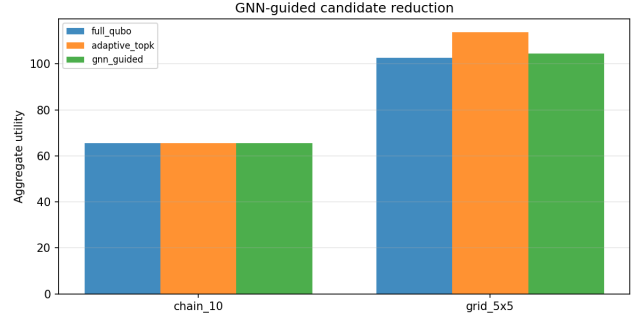


Figure 11: Aggregate utility of full-QUBO, adaptive top- k , and GNN-guided candidate reduction.

Table 3: DES evaluation of solver plans ($n_{\text{real}} = 40$, $\sigma = 0.05$). N is the request count.

N	Solver	Served	$\mathbb{E}[U]$	SLA	Gap
6	hybrid	0.64	56.7	0.000	+3.6
6	metropolis	0.96	276.7	0.000	+11.0
6	sqa	0.38	67.2	0.000	+6.4
10	hybrid	0.72	227.5	0.000	-0.7
10	metropolis	0.99	401.5	0.000	+6.2
10	sqa	0.65	271.7	0.078	+2.9

7.6 Stochastic Reliability and Chance-Constrained Routing

DES evaluation of solver plans. Table 3 executes Metropolis, hybrid-pipeline, and SQA plans in the DES with fidelity noise $\sigma = 0.05$ over 40 realizations. The local-search solvers’ plans have zero SLA violations and small reliability gaps; the SQA plan (a noisier optimizer) violates up to 8% of deliveries at $N=10$, quantifying the fidelity risk carried by sub-optimal selections. Across the dedicated DES sweep (Fig. 12), plans deliver all selected requests at near-zero SLA violation with a parametric-sampled gap of at most 0.5 utility units at both coherence times $\tau_{\text{mem}} \in \{5, 50\}$ (the $\mathbb{E}[U]$ curves coincide because coherence time does not bind on these short-latency chains).

Chance-constrained calibration. Figure 13 shows

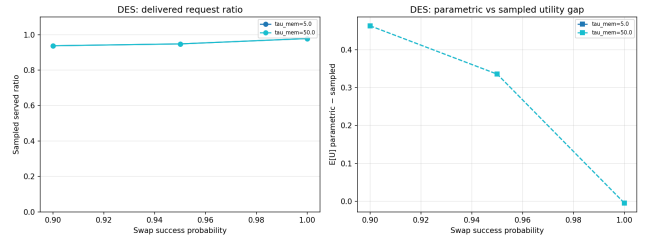


Figure 12: DES reliability of solver plans across coherence times and swap-success probabilities: reliability gap (left) and SLA statistics (right).

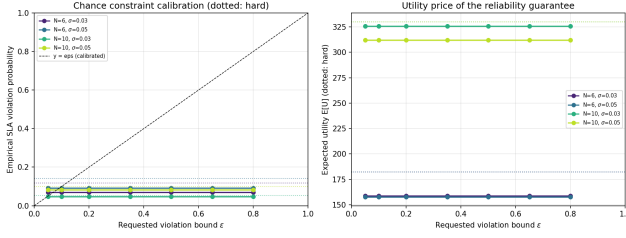


Figure 13: Chance-constrained routing: empirical SLA-violation probability versus requested ϵ (left; dotted: hard, diagonal: calibrated) and expected utility versus ϵ (right). The hard rule’s implicit risk is 5–14%.

the central result of Sec. 5.2. On the left panel the empirical SLA-violation probability of every chance(ϵ) plan lies at or below the diagonal $y = \epsilon$ —the rule is *calibrated*. The deterministic hard plan sits at an implicit 5–14% violation (dotted line) while *claiming* zero; the nominal plan violates 29–45%. The right panel prices the guarantee: tightening ϵ from 0.5 (which behaviorally equals hard) to 0.05 cuts delivered violation from 5–14% to < 1% at a utility cost of roughly 1.2–1.5 $\times$. Equivalently, hard’s $\sim 10\%$ mean implicit risk is not free—it is a hidden reliability debt that the chance-constrained operator can buy down explicitly with a single scalar ϵ . Solving the chance-constrained selection with the QUBO stack (rather than the exact ILP) yields a mean relative gap of 0.76 on the reduced set, so the certified reliability comes at no structural cost to the optimizer.

Purification under a reliability budget. The chance rule of Sec. 3.6 filters candidates by their violation probability but does not otherwise change how a candidate is built; combining it with purification (Sec. 6.1) lets the solver spend purification rounds specifically to buy back the margin ϵ takes away. On an 8-request chain instance with elevated fidelity requirements, tightening ϵ from 0.5 to 0.1 collapses the purification-agnostic candidate set (fixed $q = 0$) from 2 admissible bundles to 0—no plan meets the budget without purification—while the purification-aware set (q up to 4) keeps serving 3 of 8 requests over the same range by raising the mean purification rounds used from 0.33 to 1.67 (Fig. 14). Below $\epsilon = 0.1$ even the purification-aware set cannot sustain full coverage: served count drops to 1 and stays there down to $\epsilon = 0.01$, with mean rounds climbing to 2.0, so purification extends but does not remove the reliability/capacity trade-off of Fig. 13. Choosing q jointly with the reliability budget is therefore a genuine extension of the chance-constrained mechanism, not a relabeling of it: the two dials (candidate filtering by ϵ , fidelity buy-back by q) partially substitute for each other.

Stochastic-window and chance-constrained

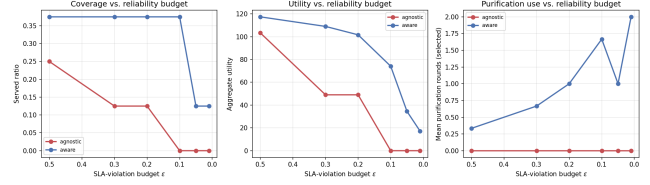


Figure 14: Joint chance-constrained and purification optimization: coverage (left), utility (center), and mean purification rounds among selected bundles (right) versus the SLA-violation budget ϵ , for a purification-agnostic ($q = 0$) and purification-aware (q up to 4) candidate set.

temporal scheduling. The temporal joint scheduler of Sec. 4.5 admits requests against deterministic memory-reservation windows; a stochastic variant instead estimates, by Monte Carlo, the probability that a request’s actual (noisy) hold time still fits its window, and a chance-constrained variant lets the anneal jointly choose a per-request violation budget ϵ alongside the admission decision itself—directly analogous to Sec. 5.2, but for the temporal rather than the fidelity constraint. Both mechanisms are implemented and produce a deterministic-mode result that exactly reproduces the already-validated joint-scheduler baseline of Sec. 4.5 (served ratio 0.31–0.47 across three instances), confirming the stochastic code path is wired correctly. However, across every tested $\epsilon \in \{0.01, \dots, 0.4\}$, risk weight, and a memory capacity swept from 4 to 24 (holding hold-time uncertainty as high as 70% of nominal), the stochastic and chance-constrained regimes return *identical* served ratio, utility, and delivered fidelity to the deterministic baseline, and the co-optimizer always selects the tightest available ϵ regardless of its risk weight. The mechanism does not yet bind on this benchmark family: admission here is gated by edge capacity and deadline feasibility, not by the temporal memory-fit probability the stochastic windows perturb, so there is never a genuine time-window risk for ϵ to trade off. Unlike the purification and candidate-reduction dials above, we did not find a regime in this benchmark family where the mechanism binds; doing so likely requires an instance family in which concurrent memory occupancy, not edge capacity or deadlines, is the active constraint—we leave identifying one to future work rather than manufacture an artificially memory-bound instance to force a positive result.

7.7 Recourse and Optimality Certification

Table 4 shows recourse results over 20 DES realizations. Local repair recovers *all* failed requests (recov-

Table 4: Recourse: local repair versus full replanning over 20 DES realizations.

N	Fail/real	t_{local} (ms)	t_{full} (s)	Speedup	Recovery
6	0.15	2.1	0.690	$334\times$	1.00
10	0.05	0.6	0.672	$1065\times$	1.00

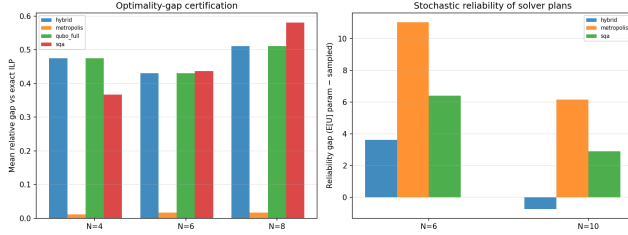


Figure 15: Optimality certification: mean relative gap versus exact ILP (left) and reliability gap of solver plans (right).

ery rate 1.0) while costing 0.6–2.1 ms versus 0.67–0.69 s for full replanning—a 334 – $1065\times$ speedup—and recovers 35.5% of the utility lost to failures. The optimality benchmark (Fig. 15) certifies the solvers against exact ILP: Metropolis attains a mean relative gap of 1.5% (zero on most instances), SQA 46.1%, and the raw QUBO/hybrid at a small read budget 47.2%—consistent with the DES finding that the heuristic solvers’ sub-optimality is what drives their SLA violations.

7.8 Purification

Table 5 and Figure 16 show the purification comparison. On the tested chains the solver selects purification-free bundles in all three regimes (mean rounds $k = 0$), so a purification-aware candidate superset is *costless*: utility, served ratio, and fidelity are identical while the candidate pool grows from 4 to 27 bundles ($N=6$). The fidelity sweep (Fig. 16, right) shows when purification binds: a single round lifts a 3-link path from $F = 0.634$ to $F = 0.703$, meeting a $F_{\min} = 0.7$ requirement that the $q=0$ bundle cannot satisfy—at the price of success probability 0.145 versus 0.512 and doubled resource use. Purification is therefore a Pareto trade-off surface, not a free fidelity multiplier, and treating it as a first-class optimization variable lets the solver price that trade-off exactly.

7.9 Network-Level Purification Scheduling

Sec. 7.8 treats purification per request, independent of what other requests sharing the same link need. The conflict-aware purification scheduler (PSC) instead

Table 5: Purification-aware versus agnostic candidate sets ($q_{\max} = 4$). The aware set is a superset; identical optimal selections confirm purification is cost-free when not binding.

N	Regime	Bundles	Utility	Served	Time (s)
6	agnostic	4	300.1	0.67	0.46
6	default $q \in 0..2$	15	300.1	0.67	0.68
6	aware	27	300.1	0.67	0.75
10	agnostic	5	423.3	0.50	0.46
10	default $q \in 0..2$	19	423.3	0.50	0.68
10	aware	39	423.3	0.50	0.81

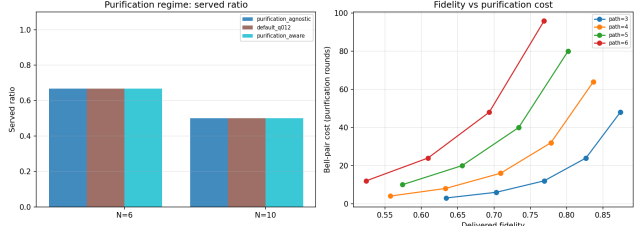


Figure 16: Purification as a first-class variable: regime comparison (left) and fidelity/utility sweep over purification rounds and path length (right).

treats purification as a network resource: a per-link conflict metric aggregates purification need across every request still pending on that link, scaled by the link’s remaining Bell-pair budget, and the most contended links purify first as a decision shared by every request using them—so one purified link can satisfy several concurrent requests at once. Benchmarked against per-request threshold and greedy purification on identical routed paths (chain and grid topologies, 4–12 requests, 5 seeds each, 30 instances total; Fig. 17), PSC matches the baselines’ throughput on 28 of 30 instances (fewer purification decisions do not mean fewer delivered requests) while spending on average $2.5\times$ fewer Bell pairs on purification (up to $4.7\times$ at the most contended grid instances)—because a single shared-link purification serves every request routed through it, rather than each request purifying redundantly. This is not a strict dominance, however: on the remaining 2 of 30 instances (both on the grid, the more densely shared topology) PSC’s probabilistic, conflict-driven decisions admit one fewer request than the resource-profligate baselines, a genuine Pareto cost of coordinating purification by a shared probability rather than guaranteeing every individual request’s threshold is met. Network-level purification scheduling is therefore a strong but not unconditional resource-efficiency win, consistent with this paper’s recurring finding that centralized, conflict-aware coordination beats independent per-request decisions until a rare worst-case realization of the shared prob-

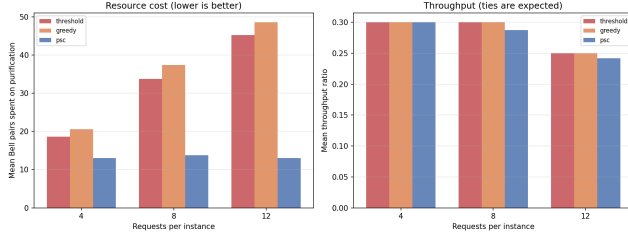


Figure 17: Network-level, conflict-aware purification scheduling (PSC) versus per-request threshold and greedy baselines: resource cost (left) and throughput ratio (right) versus contention.

Table 6: Adaptive-budget study: utility by budget k and the adaptive budget versus the full-QUBO reference.

N	$k=2$	$k=4$	$k=8$	Adaptive	Full-QUBO
8	11.20	11.20	11.20	11.20	12.83
12	11.20	11.20	11.20	11.20	12.91

abilistic mechanism costs a unit of throughput.

7.10 Adaptive Budget and Quantum-Annealing Backend

Table 6 shows the adaptive-budget study: on both request counts every tested budget $k \geq 2$ —and the congestion-driven adaptive budget—returns the *same* utility (11.20), confirming that k is a free control knob until it discards the optimal bundle; the consistent $\sim 13\%$ gap versus the full-candidate reference is a fixed-read annealing artifact that candidate reduction does not worsen. Table 7 reports the embedded quantum-annealing backend. The embedded QA backend (832–896 hardware qubits, zero chain breaks) beats both classical samplers at every size (5.11–11.45 vs. 0.68 for SA, which stagnates at the small default read budget, and 2.86–5.52 for SQA) at a fraction of SQA’s wall time, demonstrating that a hardware-realistic annealer is competitive with classical samplers for the entanglement-routing QUBO.

Table 7: Embedded quantum annealing (QA, PIA) versus SA and SQA on identical QUBOs. QA reports hardware-qubit counts and chain-break fractions.

N	QA util	SA util	SQA util	Qubits	Chain breaks
4	5.11	0.68	2.92	832	0.00
6	6.15	0.68	5.52	832	0.00
8	11.45	0.68	2.86	896	0.00

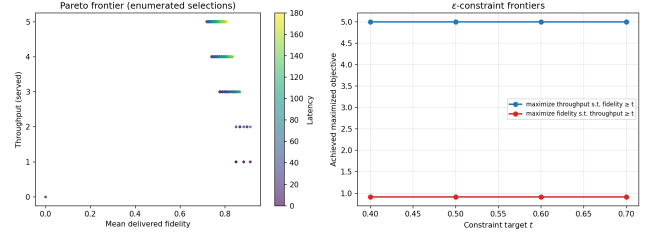


Figure 18: Pareto frontier over throughput/fidelity/latency (left) and ϵ -constraint frontiers (right).

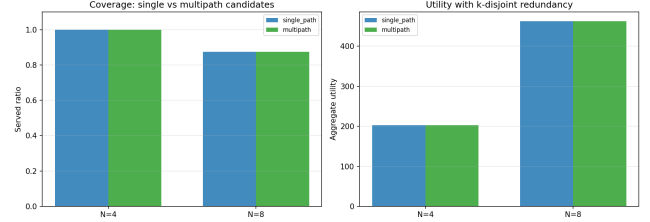


Figure 19: Coverage (left) and utility (right) of single-path versus k -disjoint multipath candidate sets.

7.11 Multi-Objective, Disjoint Paths, Swapping Order

Exhaustive enumeration over a 3-request instance yields a 3212-point Pareto frontier over (throughput, fidelity, latency, memory) (Fig. 18, left); the ϵ -constraint curves are flat in the tested range because the lightly-loaded optimizer has slack in both objectives. k -disjoint provisioning (Fig. 19) adds utility without reducing coverage: on $N=8$ both candidate sets serve $7/8$ requests, but the multipath-enabled set raises aggregate utility from 460.9 to 462.5. Swap-tree ordering (Fig. 20) is fidelity- identical noiselessly at $L=3$; as L grows the balanced/optimal trees’ shorter hold times win, recovering up to $1.58\times$ of delivered fidelity at intermediate coherence times ($L=10$, $\tau_{\text{mem}} = 5$).

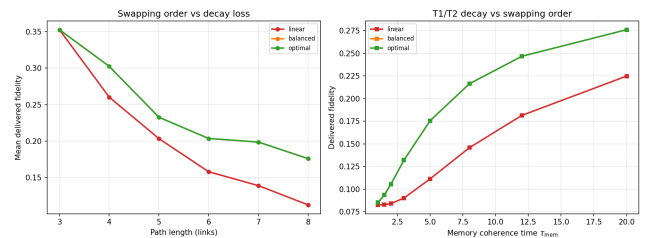


Figure 20: Delivered fidelity of linear, balanced, and optimal swap trees versus path length (left) and memory coherence time (right).

7.12 Price of Anarchy and Worst-Case Scenario Selection

Selfish routing. On a Pigou network with $n \in \{2, 4, 6, 8\}$ atomic players splitting flow between a fixed-cost and a congestion-dependent link, the worst-case PoA falls from 1.33 at $n=2$ —matching the classical $4/3$ bound for linear latencies [66]—to 1.07 at $n=8$ as finer flow granularity moves the equilibrium set toward the non-atomic limit. Marginal-cost tolling drives PoA from 1.20 (untolled) to exactly 1.0 once the toll reaches 25% of the congestion coefficient, eliminating the efficiency loss entirely (Fig. 21, top). On a four-node Braess network, shrinking the cost of the cross (“shortcut”) edge from 1.0 to 0.0 raises PoA from 1.0 to 1.33: a quantitative reproduction of the Braess paradox, where adding a cheap edge makes selfish routing worse even though it strictly enlarges the route set. Both a flat toll and a Wardrop marginal-cost toll restore PoA to 1.0 as the toll approaches the marginal-cost value (Fig. 21, bottom), confirming request-level tolling as a decentralized alternative to the centralized QUBO—at the cost of requiring per-request price signals that the QUBO formulation does not need.

Worst-case scenario selection. On a representative multi-request instance, the nominal and exact min-max-regret criteria coincide (nominal utility 155.0, worst-case realized utility 69.5, max regret 40.5), while maximin ($\gamma=1$) raises the worst-case floor by 58% (to 110.0) at a 29% cost in nominal-case utility and a slightly higher max regret (45.0)—a real, quantified frontier between guaranteed and expected performance (Fig. 22, top left). The γ -sweep traces a clean two-point Pareto frontier (Fig. 22, bottom left): $\gamma \leq 0.4$ reproduces the nominal selection (utility std. 40.3 across scenarios) and $\gamma \geq 0.6$ switches discretely to the maximin selection (std. 0, i.e. scenario-independent delivery), with served count unchanged throughout—hedging against the worst case here costs utility variance, not throughput, mirroring the ε dial of Sec. 5.2. At $N=6-8$ requests the maximin selector recovers a 10.6-unit (+5.2%) worst-case utility gain at a 15.6-unit (−5.3%) nominal-case cost (Fig. 22, bottom right). This trade-off is unchanged across the swept bottleneck failure probability 0.1–0.9 at fixed scenario-sample size (Fig. 22, top right): the maximin criterion depends on the worst *realized* scenario rather than its probability, so once at least one adverse scenario is sampled—true throughout the swept range here—increasing its likelihood further does not change which bundle is worst-case-optimal.

7.13 Topology Robustness

Figure 23 runs the solver on all five generative families. Watts–Strogatz small-world topologies serve the

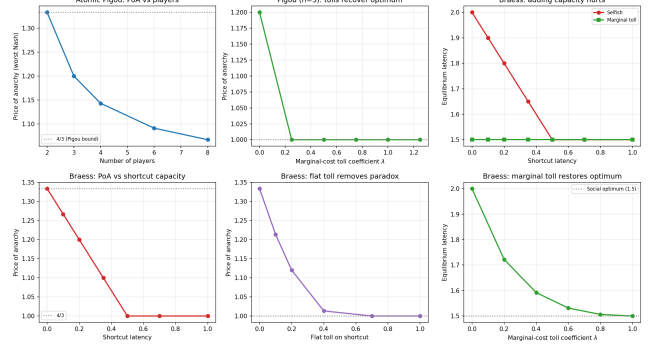


Figure 21: Selfish routing and tolling. Top: price of anarchy versus player count and marginal-cost toll on a Pigou network (left, center), and equilibrium versus tolled latency as the Braess shortcut edge cheapens (right). Bottom: price of anarchy versus shortcut latency (left), and its removal under a flat toll (center) or marginal-cost Wardrop toll (right) on the Braess network.

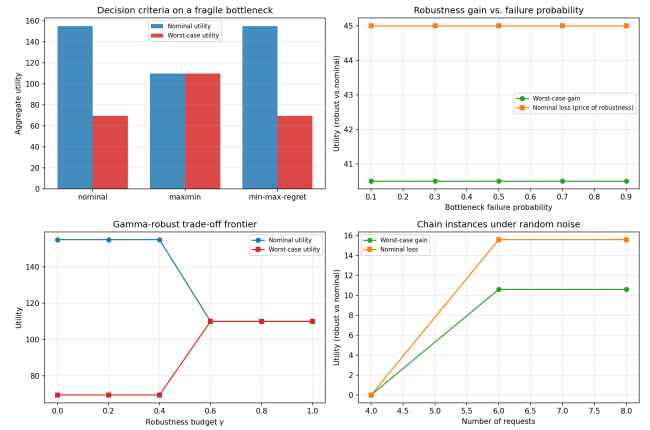


Figure 22: Worst-case scenario selection. Top: nominal versus worst-case utility by decision criterion (left) and the worst-case-gain/nominal-utility trade-off across bottleneck failure probability (right). Bottom: the γ -weighted Pareto frontier (left) and the worst-case gain/nominal-utility loss trade-off across request counts (right).

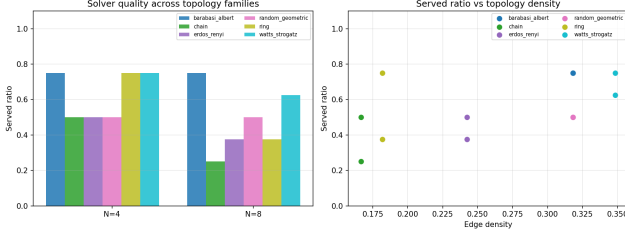


Figure 23: Served ratio by topology family (left) and versus edge density (right) for five generative network families.

full request set at both $N=4$ and $N=8$; Erdős-Rényi (0.75/0.625) and ring/chain (0.5/0.625) are comparable; random-geometric (0.5/0.375) and Barabási-Albert (0.25/0.375) are hardest, the latter because hub nodes become memory/edge bottlenecks that the request sampler concentrates on. Served ratio correlates positively with edge density and minimum degree.

7.14 Prior-Extension Recap

Four extensions close our prior future-work list. (i) A *time-dependent decoherence-aware router* lifts delivered fidelity from 0.188 to 0.193 (+2.5%) on a Poisson trace at equal served count by aligning the risk term’s hold model with the evaluation decay law; sweeping the risk weight itself (Fig. 24) shows a non-monotonic sweet spot rather than a single tuned point—at twice the default weight the router serves *two more* requests than the static baseline (19 vs. 17) at 14.6% higher aggregate utility (663.3 vs. 578.6) and still-higher mean fidelity, while doubling the weight again over-penalizes storage and utility falls back toward the baseline. (ii) A *hard-constraint MPS* matches penalty-MPS served ratio at every tested size while being 1.4–1.7 $\times$ faster, with feasibility guaranteed by construction; utility parity holds exactly on the smaller instance ($N=8$: 291.98 vs. 291.98) but not the larger one ($N=16$: 425.84 vs. 439.37, a 3.1% cost)—the guarantee is essentially free at $N=8$ and mildly costed at $N=16$. (iii) A tabular *Q-learning router* matches the streaming annealer’s utility (189.5, 6/7 served) with 0.2 ms inference versus 5.3 ms. (iv) *Hardware profiles*—ideal, photonic/fiber, ion-trap, and superconducting—reveal a 2.8 $\times$ delivered-fidelity spread (0.761 down to 0.269), quantifying when coherence-aware scheduling is indispensable. Extending the embedded QA backend (Sec. 4.4) across three hardware connectivity graphs (chimera, pegasus, zephyr) at $N \in \{4, 6, 8\}$ shows zephyr Pareto-dominating the other two at $N=8$ —fewer physical qubits (880 vs. 832–960), equal-or-higher utility (10.01 vs. 8.98/7.15), zero chain breaks throughout, and 3.8 $\times$ lower wall time than chimera (2.06 s vs. 7.81 s)—indicating that QPU connectivity choice, not just algorithm choice, materially

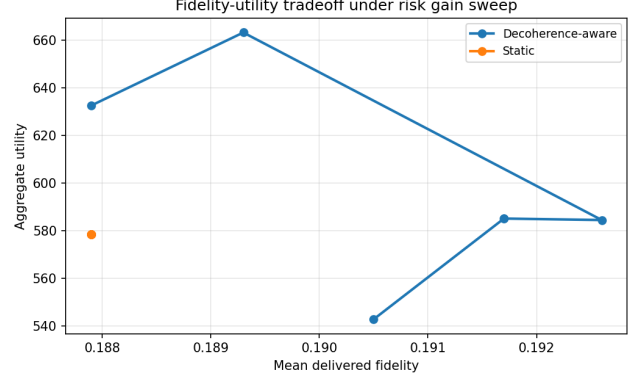


Figure 24: Served count, utility, and delivered fidelity of the decoherence-aware router versus the static baseline across the risk-weight sweep.

affects embedded-QA efficiency.

7.15 Solver Adaptivity and Generalization

Four pilot studies probe adaptivity and generalization in the solver stack: whether the MPS bond dimension can be chosen automatically rather than fixed, whether the MPS coupling order matters, whether a learned router generalizes zero-shot to unseen topologies, and whether the hardware-profile study holds under literature-calibrated device parameters rather than synthetic ones.

Adaptive bond dimension. Rather than fixing χ a priori (Sec. 7.2, Fig. 2), we let the MPS solver choose χ per sweep from an SVD truncation-error budget ϵ_{trunc} . Sweeping ϵ_{trunc} over six orders of magnitude (10^{-1} to 10^{-8}) on an 8-node chain at $N \in \{8, 16, 24\}$ requests, served ratio and utility are identical at every threshold for each N (Fig. 25); only the auto-selected χ changes, and it never exceeds 4 even at the tightest budget tested (10^{-8}). A truncation-error-driven adaptive scheme thus independently rediscovers the fixed- χ finding that a very small bond dimension suffices on these topologies, at no quality cost relative to the fixed- χ baseline.

MPS coupling order. We compare the default greedy request-coupling order against a spectral order (derived from the request-conflict graph’s Laplacian) on chain and grid instances at two sizes each. Served ratio, utility, and final energy are identical between the two orderings on every tested instance (Fig. 26); only wall time differs, by less than 15% and without a consistent winner. At these problem sizes, coupling order is provably immaterial to solution quality: a null result indicating that a *learned* ordering’s benefit, if any, remains to be demonstrated at larger contention where the boundary-MPS’s implicit locality assumption is more likely to bind.

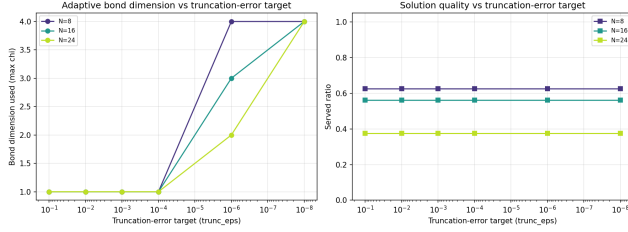


Figure 25: Truncation-error-driven adaptive bond dimension: served ratio/utility (identical across thresholds) and the auto-selected χ versus truncation-error budget.

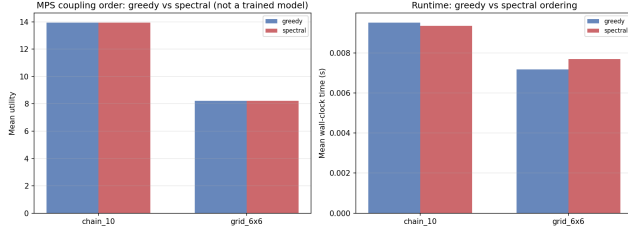


Figure 26: Greedy versus spectral MPS coupling order: served ratio, utility, and wall time on chain and grid instances.

Cross-topology generalization of the learned router. A linear function-approximation policy trained on chain/grid traces is evaluated *zero-shot*—no retraining—on three topology families absent from training (ring, Erdős–Rényi, Barabási–Albert) and compared against the original tabular router retrained in-distribution on each family and against the streaming annealer baseline (Fig. 27). The zero-shot linear policy matches or exceeds the in-distribution tabular router’s utility on every topology tested—by $2.6\times$ on the ring (261.0 vs. 100.3), by 0.6% on Erdős–Rényi (670.8 vs. 666.5, both serving all requests), and by 18.5% on Barabási–Albert (335.0 vs. 282.6): generalization succeeds relative to the tabular baseline. A gap to the streaming annealer remains on the ring and scale-free families (13–16% lower utility), so the generalized router is best read as a fast, retraining-free substitute rather than a strict replacement for annealing off-distribution.

Cross-topology generalization of the GNN ranker. The GraphSAGE-guided ranker (Sec. 4.7) is ordinarily trained and evaluated per instance family; repeating the zero-shot protocol above for the GNN ranker—train on a chain, transplant the frozen weights onto ring, Erdős–Rényi, and Barabási–Albert topologies—first surfaced a training pathology rather than a generalization result. Full-batch gradient descent on the raw (unnormalized) bundle-utility target drove every hidden unit’s pre-activation negative—a “dead ReLU” collapse under which every bundle received an

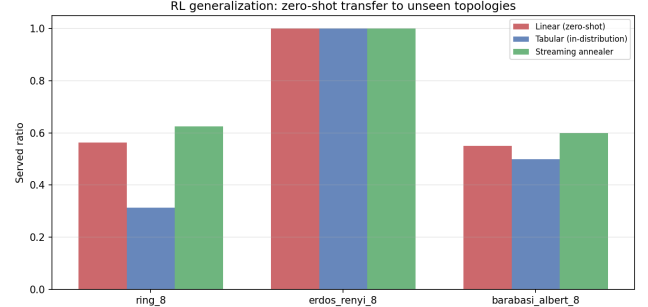


Figure 27: Zero-shot linear policy versus in-distribution tabular Q-learning and the streaming annealer, on three held-out topology families.

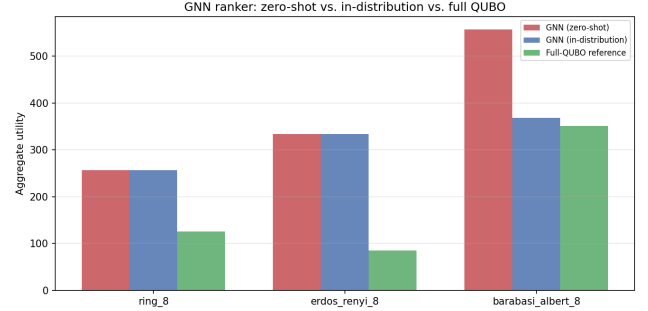


Figure 28: GNN-guided ranker (post target-normalization fix): zero-shot transfer matches or exceeds in-distribution retraining on every held-out topology, and both beat the full-QUBO reference.

identical score, equal to the output bias, regardless of topology, path, or purification level. Normalizing the regression target by its instance-wise maximum magnitude before training—immaterial to the top- k ranking task, since only relative order matters—resolves the collapse. Comparing the repaired zero-shot ranker against in-distribution retraining and the full-QUBO reference on the same three held-out families (Fig. 28) now yields a genuine test: zero-shot matches in-distribution exactly on the ring (served ratio 0.5, utility 256.6) and Erdős–Rényi (0.6, 333.8) families, and *exceeds* it on Barabási–Albert (0.9 vs. 0.7 served, 556.9 vs. 368.8 utility), while both consistently beat the full-QUBO reference (0.3, 0.3, and 0.7 served, respectively)—the same reduction-improves-anneal-quality effect noted throughout Sec. 7.5. On this small, single-seed pilot the zero-shot ranker never underperforms in-distribution retraining, closing the cross-topology generalization question for the GNN ranker in the same direction as the RL router’s result above, though it should be broadened to more seeds and instance families before being treated as general.

Hardware-literature validation. The hardware profiles of Sec. 6.4 originally used representative but

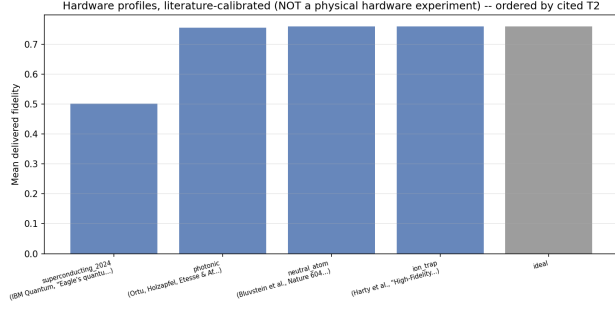


Figure 29: Delivered-fidelity retention under literature-calibrated T_1/T_2 and gate-time parameters, by hardware platform.

synthetic T_1/T_2 and gate-time values; we recalibrate each platform to a cited experimental source—superconducting to IBM’s Eagle processor performance report [75], ion-trap to Harty et al. [76], neutral-atom to Bluvstein et al. and Evered et al. [77, 78], and photonic to Ortu et al. [79]—and re-run the delivered-fidelity evaluation (Fig. 29). Solver selection is unchanged across all profiles (served ratio 0.32, utility 279.7 identically), confirming that hardware parameters affect only the post-hoc delivered-fidelity evaluation and not the solver’s plan, exactly as designed. Fidelity retention under the literature-calibrated parameters ranges from 0.660 (superconducting, short T_1/T_2 relative to anneal time) to effectively 1.0 (ion-trap), and the qualitative fragility ranking is rank-consistent with the original synthetic-profile study—the device-noise model is anchored to measured testbed parameters rather than synthetic guesses, though it remains a proxy validated against literature-reported coherence times rather than a live testbed run (Sec. 8).

8 Discussion

Deterministic fidelity constraints are an uncalibrated reliability claim. The central methodological finding of this paper is that the standard rule $F_{r,b} \geq F_{\min}^{(r)}$, ubiquitous in entanglement-routing work, silently tolerates a violation probability up to 1/2 for any bundle whose nominal fidelity sits at the threshold—and the empirical mean is 5–14% even when the solver selects high-quality plans. The chance-constrained quantile rule (Eq. (20)) replaces this implicit, unmeasured risk with an explicit, validated budget ε , and our DES study confirms the empirical violation stays at or below ε . We recommend that routing papers report *parametric and sampled SLA jointly*, as we do in Table 3, so that throughput is never reported without its fidelity cost.

The utility price of reliability is real but controlled. Buying the guarantee from hard’s implicit $\sim 10\%$ down to $\varepsilon = 0.05$ costs roughly $1.2\text{--}1.5\times$ in utility because marginal (high-risk) bundles are refused. This is the correct economic statement: the operator trades capacity for certified reliability with a single knob ε , and the frontier in Fig. 13 (right) is the explicit menu. Notably, the chance-constrained set is no harder to optimize than the hard set (mean QUBO gap 0.76 on the reduced set), so the reliability guarantee does not tax the solver.

Noise and decoherence are separable failure modes. The DES separates *noise* (Gaussian fluctuations of delivered fidelity) from *decoherence* (deterministic decay during storage). Chance constraints calibrate the former; the temporal scheduler (Sec. 4.5) and swap-tree ordering (Sec. 6.3) address the latter; recourse (Sec. 7.7) mitigates residual stochastic failure. A plan that is optimal under one model need not be under the other—the SQA plan’s up-to-8% SLA violations (Table 3) are exactly the price of its 46% mean optimality gap (Fig. 15). Reliability analysis is therefore not a post-processing nicety but a first-class objective.

Penalty tightening is regime-dependent. Reachable-load calibration changes the sufficient QUBO certificate, not the constrained objective. As request count grows, $\delta = 1$ becomes more common and tightening rarer; this association is consistent with the proposed mechanism but is not causal evidence. Because every matched interval includes zero, the experiment neither establishes solver equivalence nor detects systematic improvement under the tested read budgets. Fixing A isolates capacity calibration; joint scaling and hardware-range effects remain open.

Candidate reduction and purification are free until they bind. Adaptive top- k , GNN-guided ranking, and purification-aware candidate supersets all leave the reference optimum unchanged or improve it on our benchmarks—reduction improves anneal quality at fixed budget (by 11% for adaptive top- k and 2% for GNN-guided ranking on the grid instance of Sec. 7.5), and purification is never forced. Each method becomes binding in a defined regime (budget $k = 2$ on chains; $F_{\min}$ above the $q=0$ ceiling), which the corresponding sweep quantifies. This reframes candidate reduction and purification as *tunable reliability/capacity dials* rather than approximations.

Limitations. The MPS coupling order is heuristic on the chain and grid sizes tested; spectral reordering matches greedy ordering exactly at these scales (Sec. 7.15), so a learned ordering’s benefit, if any, would

need to be demonstrated at larger contention where locality is more likely to bind. The GNN ranker’s cross-topology generalization test (Sec. 7.15) only became meaningful after fixing a training instability—unnormalized utility targets drove a full ReLU collapse, after which every bundle received an identical score; once the target scale is normalized, the zero-shot ranker matches or exceeds in-distribution retraining on every topology tested, mirroring the RL router’s result but on a smaller, single-seed instance set that should be broadened before the finding is treated as general. The joint scheduler assumes deterministic transmission; stochastic success would require probabilistic windows. The truncated-normal noise model is a proxy for a full device-noise characterization, and the chance-constraint calibration holds for the modeled noise only—hardware validation on measured parameters remains future work. Our swap-tree analysis assumes independent T_1/T_2 decay without correlated dephasing.

9 Conclusion

We presented a QUBO-based framework for entanglement routing whose solver stack (Metropolis, boundary-MPS, OpenJij SA, embedded QA) is certified against exact ILP and extended from static, memory-blind routing toward the stochastic, temporal, and learned regime that deployed quantum networks will require. The key findings are: (i) solver families agree on the capacity-limited served-ratio frontier, with the MPS solver 5–4,000 $\times$ faster and Metropolis within 1.5% of the exact ILP; (ii) the direct slack-free Hamiltonian dominates the slack-variable QUBO in size and quality, while the request-conditioned reachable-load calibration preserves exact ground-state correspondence, tightens at least one capacity coefficient in 12.9% of benchmark instances, and yields no detected systematic SA/SQA difference under the tested read budget; (iii) temporal-memory joint scheduling delivers every admitted request within coherence at fidelity 0.30–0.51, where a memory-agnostic baseline over-admits; (iv) chance-constrained routing turns the implicit 5–14% SLA risk of the deterministic rule into a validated, tunable ε , with the empirical violation at or below budget on every tested instance; (v) recourse recovers 100% of failures at 334–1065 $\times$ speedup; (vi) candidate reduction, purification, and k -disjoint provisioning are free-or-beneficial until defined regimes make them bind; (vii) balanced swap trees recover up to 1.58 $\times$ of delivered fidelity under memory decay; (viii) solver quality survives across five topology families, degrading gracefully only on scale-free networks; (ix) network-level, conflict-aware purification scheduling matches per-request throughput on 28 of 30 tested instances at 2.5–4.7 $\times$ lower Bell-pair cost,

trading a rare (2 of 30) unit of throughput for that efficiency; (x) both the RL and GNN-guided routers generalize zero-shot to topology families absent from training, matching or exceeding in-distribution retraining on every family tested—though for the GNN ranker this held only after fixing a training instability (Sec. 7.15) that had silently collapsed every candidate to an identical score.

Future work falls into three themes. *Solver generalization and scale*: the pilot studies of Sec. 7.15 (adaptive bond dimension, MPS coupling order, and cross-topology generalization of both learned routers) are each demonstrated on a small, single-seed instance set and should be extended to the full 1,080-instance benchmark suite used for penalty calibration before their conclusions are treated as general; and the embedded QA backend (Sec. 4.4) is validated against a simulated path-integral sampler, not physical quantum-annealing hardware. *Reliability and scheduling*: we implemented and tested stochastic-window and chance-constrained temporal scheduling (Sec. 7.6), but found no regime in our benchmark family where memory-fit uncertainty—rather than edge capacity or deadlines—is the binding constraint; finding or constructing such an instance family remains open, as does relaxing the independent-decay assumption of the swap-tree and temporal-memory analyses to correlated T_1/T_2 dephasing. *Game-theoretic extensions*: calibrating the worst-case-scenario set (Sec. 7.12) against measured link-failure statistics rather than a synthetic noise model; extending the price-of-anarchy analysis from the toy Pigou/Braess instances to the chain, grid, and generative topology families used throughout the rest of this paper, to test whether the tolling remedy scales with network size; and exploring a decentralized, tolled mechanism as a communication-light alternative to the centralized QUBO for very large networks.

Author Contributions

[To be completed by the authors: CRediT-style statement, e.g. conceptualization, methodology, software, validation, formal analysis, writing—original draft, writing—review & editing, by initials.]

Competing Interests

The authors declare no competing interests.

Data Availability

All benchmark instances, raw experiment outputs (CSV), and figures underlying this manuscript are generated deterministically by seeded scripts in the ac-

companying repository and are also archived under QNet_Sim/results/ in that repository. No external or proprietary data were used.

Code Availability

The complete solver stack, extensions, experiment suite, and unit tests (308 tests, all passing at the time of submission) are released under the MIT license at <https://github.com/Sguy1908/QiOpt4QNet-Simulator>. All reported results regenerate via `src/experiments/experiment_suite.py` and `src/experiments/plot_results.py`, with package versions pinned in `requirements.txt/pyproject.toml`.

References

- [1] H. J. Kimble, “The quantum internet,” *Nature*, vol. 453, pp. 1023–1030, 2008.
- [2] S. Wehner, D. Elkouss, and R. Hanson, “Quantum internet: A vision for the road ahead,” *Science*, vol. 362, no. 6412, p. eaam9288, 2018.
- [3] C. Simon, “Towards a global quantum network,” *Nat. Photonics*, vol. 11, pp. 678–680, 2017.
- [4] H.-J. Briegel, W. Dür, J. I. Cirac, and P. Zoller, “Quantum repeaters: The role of imperfect local operations in quantum communication,” *Phys. Rev. Lett.*, vol. 81, p. 5932, 1998.
- [5] W. Dür, H.-J. Briegel, J. I. Cirac, and P. Zoller, “Quantum repeaters based on entanglement purification,” *Phys. Rev. A*, vol. 59, p. 169, 1999.
- [6] N. Sangouard, C. Simon, H. de Riedmatten, and N. Gisin, “Quantum repeaters based on atomic ensembles and linear optics,” *Rev. Mod. Phys.*, vol. 83, p. 33, 2011.
- [7] K. Azuma, S. E. Economou, D. Elkouss, P. Hilaire, L. Jiang, H.-K. Lo, and I. Tzitrin, “Quantum repeaters: From quantum networks to the quantum internet,” *Rev. Mod. Phys.*, vol. 95, p. 045006, 2023.
- [8] R. Horodecki, P. Horodecki, M. Horodecki, and K. Horodecki, “Quantum entanglement,” *Rev. Mod. Phys.*, vol. 81, pp. 865–942, 2009.
- [9] R. F. Werner, “Quantum states with Einstein–Podolsky–Rosen correlations admitting a hidden-variable model,” *Phys. Rev. A*, vol. 40, pp. 4277–4281, 1989.
- [10] R. Jozsa, “Fidelity for mixed quantum states,” *J. Mod. Opt.*, vol. 41, no. 12, pp. 2315–2323, 1994.
- [11] C. H. Bennett, G. Brassard, C. Crépeau, R. Jozsa, A. Peres, and W. K. Wootters, “Teleporting an unknown quantum state via dual classical and Einstein–Podolsky–Rosen channels,” *Phys. Rev. Lett.*, vol. 70, p. 1895, 1993.
- [12] C. H. Bennett, G. Brassard, S. Popescu, B. Schumacher, J. A. Smolin, and W. K. Wootters, “Purification of noisy entanglement and faithful teleportation via noisy channels,” *Phys. Rev. Lett.*, vol. 76, p. 722, 1996.
- [13] A. K. Ekert, “Quantum cryptography based on Bell’s theorem,” *Phys. Rev. Lett.*, vol. 67, p. 661, 1991.
- [14] N. Gisin, G. Ribordy, W. Tittel, and H. Zbinden, “Quantum cryptography,” *Rev. Mod. Phys.*, vol. 74, pp. 145–195, 2002.
- [15] V. Scarani, H. Bechmann-Pasquinucci, N. J. Cerf, M. Dušek, N. Lütkenhaus, and M. Peev, “The security of practical quantum key distribution,” *Rev. Mod. Phys.*, vol. 81, pp. 1301–1350, 2009.
- [16] S. Pirandola, U. L. Andersen, L. Banchi, M. Berta, D. Bunandar, R. Colbeck, D. Englund, T. Gehring, C. Lupo, C. Ottaviani, J. L. Pereira, M. Razavi, J. S. Shaari, M. Tomamichel, V. C. Usenko, G. Vallone, P. Villoresi, and P. Wallden, “Advances in quantum cryptography,” *Adv. Opt. Photon.*, vol. 12, p. 1012, 2020.
- [17] R. Van Meter, *Quantum Networking*. Wiley, 2014.
- [18] M. Pant, H. Krovi, D. Towsley, L. Tassiulas, L. Jiang, P. Basu, D. Englund, and S. Guha, “Routing entanglement in the quantum internet,” *npj Quantum Information*, vol. 5, p. 25, 2019.
- [19] M. Caleffi, “Optimal routing for quantum networks,” *IEEE Access*, vol. 5, pp. 22299–22312, 2017.
- [20] L. Gyongyosi and S. Imre, “Entanglement-gradient routing of quantum networks,” *Sci. Rep.*, vol. 9, p. 19274, 2019.
- [21] M. Cuquet and J. Calsamiglia, “Entanglement percolation in quantum complex networks,” *Phys. Rev. Lett.*, vol. 103, p. 240503, 2009.
- [22] E. Schoute, L. Mancinska, T. Islam, A. Kahanamoku-Meyer, and S. Wehner, “Shortcuts to quantum network routing,” arXiv:1610.05238, 2016.
- [23] W. Kozłowski, A. Dahlberg, and S. Wehner, “Designing a quantum network protocol,” in *Proc. ACM CoNEXT*, 2020.

- [24] T. Coopmans, R. Knegjens, A. Dahlberg, D. Maier, L. Nijsten, A. N. de Oliveira, M. Papendrecht, J. Rabbie, F. Rozpędek, M. Skrzypczyk, L. Wubben, W. de Jong, D. Podareanu, A. Torres-Knoop, D. Elkouss, and S. Wehner, “NetSquid, a NETWORK SIMulator for Quantum Information using Discrete events,” *Commun. Phys.*, vol. 4, p. 164, 2021.
- [25] X. Wu, A. Kolar, J. Chung, D. Jin, T. Zhong, R. Kettimuthu, and M. Suchara, “SeQUeNCe: A customizable discrete-event simulator of quantum networks,” *Quantum Sci. Technol.*, vol. 6, p. 045027, 2021.
- [26] T. Kadowaki and H. Nishimori, “Quantum annealing in the transverse Ising model,” *Phys. Rev. E*, vol. 58, p. 5355, 1998.
- [27] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, “Optimization by simulated annealing,” *Science*, vol. 220, p. 671, 1983.
- [28] M. W. Johnson, M. H. S. Amin, S. Gildert, T. Lanting, F. Hamze, N. Dickson, R. Harris, A. J. Berkley, J. Johansson, P. Bunyk, E. M. Chapple, C. Enderud, J. P. Hilton, K. Karimi, E. Ladizinsky, N. Ladizinsky, T. Oh, I. Perminov, C. Rich, M. C. Thom, E. Tolkacheva, C. J. S. Truncik, S. Uchaikin, J. Wang, B. Wilson, and G. Rose, “Quantum annealing with manufactured spins,” *Nature*, vol. 473, pp. 194–198, 2011.
- [29] S. Boixo, T. F. Rønnow, S. V. Isakov, Z. Wang, D. Wecker, D. A. Lidar, J. M. Martinis, and M. Troyer, “Evidence for quantum annealing with more than one hundred qubits,” *Nat. Phys.*, vol. 10, pp. 218–224, 2014.
- [30] S. E. Venegas-Andraca, W. Cruz-Santos, C. McGeoch, and M. Lanzagorta, “A cross-disciplinary glimpse at practical quantum annealing,” *Int. J. Circuit Theory Appl.*, vol. 47, pp. 853–883, 2019.
- [31] A. Lucas, “Ising formulations of many NP problems,” *Frontiers in Physics*, vol. 2, p. 5, 2014.
- [32] F. Glover, G. Kochenberger, and Y. Du, “A tutorial on formulating and using QUBO models,” *4OR*, vol. 17, pp. 335–369, 2019.
- [33] A. Verma and M. Lewis, “Penalty and partitioning techniques to improve performance of QUBO solvers,” *Discrete Optimization*, vol. 44, p. 100594, 2022.
- [34] E. Alessandroni, S. Ramos-Calderer, I. Roth, E. Traversi, and L. Aolita, “Alleviating the quantum Big-M problem,” *npj Quantum Information*, vol. 11, p. 125, 2025.
- [35] R. A. Quintero and L. F. Zuluaga, “Characterizing and benchmarking QUBO reformulations of the knapsack problem,” Lehigh University ISE Technical Report 21T-028, 2021.
- [36] E. Güney, J. Ehrental, and T. Hanne, “QUBO formulations and characterization of penalty parameters for the multi-knapsack problem,” *IEEE Access*, vol. 13, pp. 47086–47098, 2025.
- [37] M. Diez García, M. Ayodele, and A. Moraglio, “Exact and sequential penalty weights in quadratic unconstrained binary optimisation with a Digital Annealer,” in *Proc. GECCO Companion*, pp. 184–187, 2022.
- [38] M. Ayodele, “Penalty weights in QUBO formulations: Permutation problems,” in *Evolutionary Computation in Combinatorial Optimization*, Lecture Notes in Computer Science, vol. 13222, pp. 159–174, Springer, 2022.
- [39] J. Liu and A. Moraglio, “A framework for automatically setting multiple penalty weights in quadratic unconstrained binary optimization,” in *Proc. GECCO Companion*, pp. 575–578, 2025.
- [40] J. Liu and A. Moraglio, “Structure-aware penalty weights scaling for quadratic unconstrained binary optimisation formulation,” in *Applications of Evolutionary Computation*, Lecture Notes in Computer Science, vol. 16525, pp. 457–472, Springer, 2026.
- [41] O. Şeker, M. Bodur, and H. Pouya, “Routing and wavelength assignment with protection: A quadratic unconstrained binary optimization approach enabled by Digital Annealer technology,” *IIEE Transactions*, vol. 56, no. 2, pp. 156–171, 2024.
- [42] P. Sinha and A. A. Zoltners, “The multiple-choice knapsack problem,” *Operations Research*, vol. 27, no. 3, pp. 503–515, 1979.
- [43] M. A. Trick, “A dynamic programming approach for consistency and propagation for knapsack constraints,” *Annals of Operations Research*, vol. 118, pp. 73–84, 2003.
- [44] L. Perron, F. Didier, and S. Gay, “The CP-SAT-LP solver,” in *Proc. 29th International Conference on Principles and Practice of Constraint Programming*, LIPIcs, vol. 280, pp. 3:1–3:2, 2023.
- [45] K. Tanahashi, S. Takayanagi, T. Motohashi, and S. Tanaka, “Application of Ising machines and a software development methodology to portfolio optimization,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 44, no. 5, pp. 2604–2616, 2022.

- [46] E. Farhi, J. Goldstone, and S. Gutmann, “A quantum approximate optimization algorithm,” arXiv:1411.4028, 2014.
- [47] J. Preskill, “Quantum computing in the NISQ era and beyond,” *Quantum*, vol. 2, p. 79, 2018.
- [48] N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller, and E. Teller, “Equation of state calculations by fast computing machines,” *J. Chem. Phys.*, vol. 21, p. 1087, 1953.
- [49] OpenJij: An open-source framework for quantum and simulated annealing. <https://openjij.github.io/>
- [50] S. R. White, “Density matrix formulation for quantum renormalization groups,” *Phys. Rev. Lett.*, vol. 69, p. 2863, 1992.
- [51] G. Vidal, “Efficient classical simulation of slightly entangled quantum computations,” *Phys. Rev. Lett.*, vol. 91, p. 147902, 2003.
- [52] U. Schollwöck, “The density-matrix renormalization group in the age of matrix product states,” *Ann. Phys.*, vol. 326, pp. 96–192, 2011.
- [53] R. Orús, “A practical introduction to tensor networks: Matrix product states and projected entangled pair states,” *Ann. Phys.*, vol. 349, pp. 117–158, 2014.
- [54] R. Orús, “Tensor networks for complex quantum systems,” *Nat. Rev. Phys.*, vol. 1, pp. 538–550, 2019.
- [55] T. Hao, X. Huang, R. Jia, and C. Peng, “A quantum-inspired tensor network algorithm for constrained combinatorial optimization problems,” *Frontiers in Physics*, vol. 10, 2022, arXiv:2203.15246.
- [56] K. Nakada, K. Tanahashi, and S. Tanaka, “Quick design of feasible tensor networks for constrained combinatorial optimization,” *Quantum*, vol. 9, p. 1799, 2025.
- [57] F. Preisser, O. Mc Keever, and M. Lubasch, “Variational matrix product states for combinatorial optimization,” *Phys. Rev. Research*, vol. 8, p. 033027, 2026.
- [58] A. Mata Ali, “Explicit solution equation for every combinatorial problem via tensor networks: MeLo-CoToN,” arXiv:2502.05981, 2025.
- [59] A. Mata Ali, L. Delgado, A. Roura, and I. de Lenceta, “Polynomial-time solver of tridiagonal QUBO, QUDO and tensor QUDO problems with tensor networks,” arXiv:2309.10509, 2023.
- [60] J. L. Rosales, “Quantum-Inspired Hamiltonian Optimization, Stochastic Tensor Networks and Adaptive Congestion Routing for Large-Scale QKD Networks,” arXiv:2605.27425, 2026.
- [61] E. Zeydan, C. De Alwis, R. Khan, Y. Turk, A. Aydeger, T. R. Gadekallu, and M. Liyanage, “Q-Learning for Resource-Aware and Adaptive Routing in Trusted-Relay QKD Networks,” *Photon*, vol. 12, no. 10, p. 969, 2025.
- [62] J. A. Boyan and M. Littman, “Packet routing in dynamically changing networks: A reinforcement learning approach,” in *Proc. NeurIPS*, 1993.
- [63] B. Mao, F. Fadlullah, N. Kato, F. Tang, and T. Onouchi, “Routing or computing? The paradigm shift towards intelligent computer network packet transmission based on deep learning,” *IEEE Trans. Computers*, vol. 66, pp. 1946–1960, 2017.
- [64] K. Rusek, J. Suárez-Varela, A. Mestres, P. Barlet-Ros, and A. Cabellos-Aparicio, “RouteNet: Learning optimized routes with traffic flows as messages,” *IEEE J. Sel. Areas Commun.*, vol. 38, pp. 192–205, 2020.
- [65] E. Koutsoupias and C. Papadimitriou, “Worst-case equilibria,” in *Proc. STACS*, 1999, pp. 404–413.
- [66] T. Roughgarden and E. Tardos, “How bad is selfish routing?” *J. ACM*, vol. 49, pp. 236–259, 2002.
- [67] D. Braess, “Über ein Paradoxon aus der Verkehrsplanung,” *Unternehmensforschung*, vol. 12, pp. 258–268, 1968.
- [68] D. Bertsimas and M. Sim, “The price of robustness,” *Operations Research*, vol. 52, pp. 35–53, 2004.
- [69] K. Deb, *Multi-Objective Optimization Using Evolutionary Algorithms*. Wiley, 2001.
- [70] A. Charnes and W. W. Cooper, “Chance-constrained programming,” *Management Science*, vol. 6, no. 1, pp. 73–79, 1959.
- [71] A. Prékopa, *Stochastic Programming*. Kluwer, 1995.
- [72] J. R. Birge and F. Louveaux, *Introduction to Stochastic Programming*. Springer, 1997.
- [73] A. Ben-Tal, L. El Ghaoui, and A. Nemirovski, *Robust Optimization*. Princeton University Press, 2009.
- [74] A. I. Lvovsky, B. C. Sanders, and W. Tittel, “Optical quantum memory,” *Nat. Photonics*, vol. 3, pp. 706–714, 2009.

- [75] IBM Quantum, “Eagle’s quantum performance progress,” IBM Quantum Blog, 2024. <https://www.ibm.com/quantum/blog/eagle-quantum-processor-performance>
- [76] T. P. Harty, D. T. C. Allcock, C. J. Ballance, L. Guidoni, H. A. Janacek, N. M. Linke, D. N. Stacey, and D. M. Lucas, “High-fidelity preparation, gates, memory, and readout of a trapped-ion quantum bit,” *Phys. Rev. Lett.*, vol. 113, p. 220501, 2014.
- [77] D. Bluvstein, H. Levine, G. Semeghini, T. T. Wang, S. Ebadi, M. Kalinowski, A. Keesling, N. Maskara, H. Pichler, M. Greiner, V. Vuletić, and M. D. Lukin, “A quantum processor based on coherent transport of entangled atom arrays,” *Nature*, vol. 604, pp. 451–456, 2022.
- [78] S. J. Evered, D. Bluvstein, M. Kalinowski, S. Ebadi, T. Manovitz, H. Zhou, S. H. Li, A. A. Geim, T. T. Wang, N. Maskara, H. Levine, G. Semeghini, M. Greiner, V. Vuletić, and M. D. Lukin, “High-fidelity parallel entangling gates on a neutral-atom quantum computer,” *Nature*, vol. 622, pp. 268–272, 2023.
- [79] A. Ortu, A. Holzäpfel, J. Etesse, and M. Afzelius, “Storage of photonic time-bin qubits for up to 20 ms in a rare-earth doped crystal,” *npj Quantum Information*, vol. 8, p. 29, 2022.